
Generalizing Stochastic Smoothing for Differentiation and Gradient Estimation

Felix Petersen^{1 2} Christian Borgelt^{1 3} Aashwin Ananda Mishra^{4 2} Stefano Ermon²

Abstract

We address the problem of gradient estimation for stochastic differentiable relaxations of algorithms, operators, simulators, and other non-differentiable functions. Stochastic smoothing conventionally perturbs the input of a non-differentiable function with a differentiable density with full support, smoothing it and enabling gradient estimation. Our theory starts at first principles to derive stochastic smoothing with reduced assumptions, neither requiring a differentiable density nor full support, and we present a general framework for relaxation and gradient estimation of non-differentiable black-box functions $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$. We develop variance reduction for gradient estimation from 3 orthogonal perspectives. Empirically, we benchmark 6 distributions and up to 24 variance reduction strategies for differentiable sorting and ranking, differentiable shortest-paths on graphs, differentiable rendering for pose estimation and differentiable cryo-ET simulations.

1. Introduction

The differentiation of algorithms, operators, and other non-differentiable functions has been a topic of rapidly increasing interest in the machine learning community (Berthet et al., 2020; Petersen et al., 2023; Stewart et al., 2023; Petersen, 2022; Petersen et al., 2021b; Cuturi & Blondel, 2017; Cuturi et al., 2019). In particular, whenever we want to integrate a non-differentiable operation (such as ranking) into a machine learning pipeline, we need to relax it into a differentiable form in order to allow for backpropagation. To give a concrete example, a body of recent work considered continuously relaxing the sorting and ranking operators for

tasks like learning-to-rank (Grover et al., 2019; Prillo & Eischenschlos, 2020; Cuturi et al., 2019; Blondel et al., 2020; Petersen et al., 2021a;b; 2022a; Sander et al., 2023; Vauvelles et al., 2023; Shvetsova et al., 2023). These works can be categorized into either casting sorting and ranking as a related problem (e.g., optimal transport (Cuturi, 2013)) and differentially relaxing it (e.g., via entropy-regularized OT (Cuturi et al., 2019)) or by considering a sorting algorithm and continuously relaxing it on the level of individual operations or program statements (Petersen et al., 2021b;a; Shvetsova et al., 2023; Vauvelles et al., 2023). To give another example, in the space of differentiable graph algorithms and clustering, popular directions either relax algorithms on a statement level (Petersen et al., 2021b) or cast the algorithm as a convex optimization problem and differentiate the solution of the optimization problem under perturbed parameterization (Vlastelica et al., 2020; Berthet et al., 2020; Stewart et al., 2023).

Complementary to these directions of research, in this work, we consider algorithms, operators, simulators, and other non-differentiable functions directly as black-box functions and differentially relax them via stochastic smoothing (Abernethy et al., 2016), i.e., via stochastic perturbations of the inputs and via multiple function evaluation. This is challenging as, so far, gradient estimators have come with large variance and supported only a restrictive set of smoothing distributions. More concretely, for a black-box function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, we consider the problem of estimating the derivative (or gradient) of the relaxation

$$f_\epsilon(x) = \mathbb{E}_{\epsilon \sim \mu} [f(x + \epsilon)] = \int f(x + \epsilon) \mu(\epsilon) d\epsilon \quad (1)$$

where ϵ is a sample from a probability distribution with an (absolutely) continuous density $\mu(\epsilon)$. f_ϵ is a differentiable function (regardless of differentiability properties of f itself, see Section 2 for details) and its gradient is well defined. Under limiting restrictions on the probability distribution μ used for smoothing, gradient estimators exist in the literature (Glasserman, 1990; Williams, 1992; Abernethy et al., 2016; Berthet et al., 2020).

The **contributions** of this work lie in providing more generalized gradient estimators (reducing assumptions on μ , Lemma 2.3) that exhibit reduced variances (Sec. 2.2) for

¹DiffLogic Inc, Santa Clara, CA, USA ²Stanford University, Stanford, CA, USA ³University of Salzburg, Salzburg, Austria ⁴Luminary, San Mateo, CA, USA. Correspondence to: Aashwin Ananda Mishra <aashwin@stanford.edu>.

the application of differentially relaxing conventionally non-differentiable algorithms. Moreover, we enable smoothing with and differentiation wrt. non-diagonal covariances (Thm. 2.7), outline formal requirements for f (Lem. B.1+B.2), discriminate smoothing of algorithms and losses (Sec. 2.3), and provide a k -sample median extension (Apx. C). The proposed approach is applicable for differentiation of (i) arbitrary¹ functions, which are (ii) seen as a black-box, (iii) can be called many times (primarily) at low cost, and (iv) should be smoothed with any distribution with absolutely continuous density on $\mathbb{R}$. This contrasts prior work, which smoothed (i) convex optimizers (Berthet et al., 2020; Stewart et al., 2023), (ii) used first-order gradients (Liu et al., 2019; Petersen et al., 2021a; Shvetsova et al., 2023; Cuturi et al., 2019), (iii) allowed calling an environment only once or few times in RL (Williams, 1992), and/or (iv) smoothed with differentiable densities with full support (Abernethy et al., 2016; Berthet et al., 2020; Lidec et al., 2021; Stewart et al., 2023). In machine learning, many other subfields also utilize the ideas underlying stochastic smoothing; stochastic smoothing and similar methods can be found, e.g., in REINFORCE (Williams, 1992), the score function estimator (Fu, 2006), the CatLog-Derivative trick (De Smet et al., 2024), perturbed optimizers (Berthet et al., 2020; Stewart et al., 2023), among others.

Conflict of Interest Disclosure: The authors declare that they had no financial or other substantive conflicts of interest that could be perceived to influence the work.

2. Differentiation via Stochastic Smoothing

We begin by recapitulating the core of the stochastic smoothing method. The idea behind smoothing is that given a (potentially non-differentiable) function¹ $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, we can relax the function to a differentiable function by perturbing its argument with a probability distribution: if $\epsilon \in \mathbb{R}^n$ follows a distribution with a differentiable density $\mu(\epsilon)$, then $f_\epsilon(x) = \mathbb{E}_\epsilon[f(x + \epsilon)]$ is differentiable.

For the case of $m = 1$, i.e., for a scalar function f , we can compute the gradient of f_ϵ by following and extending part of Lemma 1.5 in (Abernethy et al., 2016) as follows:

¹Traditionally and formally, only functions with compact range have been considered for f (i.e., $f : \mathbb{R}^n \rightarrow [a, b]$) (Showalter, 2010; Glasserman, 1990). More recently, i.a., Abernethy et al. (Abernethy et al., 2016) have considered the general case of functions with real range. While this is very helpful, e.g., enabling linear functions for f , this is (even without our generalizations) not always finitely defined as we discuss with the help of degenerate examples in Appendix B. There, we characterize the set of valid f leading to finitely defined f_ϵ in Lemma B.1 as well as ∇f_ϵ in Lemma B.2 in dependence on μ . We remark that, beyond discussions in Appendix B, we assume this to be satisfied by f .

Lemma 2.1 (Differentiable Density Smoothing). *Given a function¹ $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and a differentiable density $\mu(\epsilon)$ with full support on $\mathbb{R}^n$, then f_ϵ is differentiable and*

$$\begin{aligned} \nabla_x f_\epsilon(x) &= \nabla_x \mathbb{E}_{\epsilon \sim \mu} [f(x + \epsilon)] \\ &= \mathbb{E}_{\epsilon \sim \mu} [f(x + \epsilon) \cdot \nabla_\epsilon - \log \mu(\epsilon)]. \end{aligned} \quad (2)$$

Proof. For didactic reasons, we include a full proof here to support understanding the core of the method. Via a change of variables, replacing $x + \epsilon$ by u , we obtain ($d\epsilon/du = 1$)

$$f_\epsilon(x) = \int f(x + \epsilon) \mu(\epsilon) d\epsilon = \int f(u) \mu(u - x) du. \quad (3)$$

$$\begin{aligned} \text{Now, } \nabla_x f_\epsilon(x) &= \nabla_x \int f(u) \mu(u - x) du \\ &= \int f(u) \nabla_x \mu(u - x) du. \end{aligned} \quad (4)$$

Using $\nabla_x \mu(u - x) = -\nabla_\epsilon \mu(\epsilon)$,

$$\nabla_x f_\epsilon(x) = - \int f(x + \epsilon) \nabla_\epsilon \mu(\epsilon) d\epsilon. \quad (5)$$

Because $\frac{\partial \mu(\epsilon)}{\partial \epsilon} = \mu(\epsilon) \cdot \frac{\partial \log \mu(\epsilon)}{\partial \epsilon}$, we can simplify to

$$\begin{aligned} \nabla_x f_\epsilon(x) &= - \int f(x + \epsilon) \mu(\epsilon) \nabla_\epsilon \log \mu(\epsilon) d\epsilon \\ &= \mathbb{E}_{\epsilon \sim \mu} [f(x + \epsilon) \nabla_\epsilon - \log \mu(\epsilon)]. \end{aligned} \quad (6) \quad \square$$

Empirically, for a number of samples s , this gradient estimator can be evaluated without bias via

$$\nabla_x f_\epsilon(x) \triangleq \frac{1}{s} \sum_{i=1}^s [f(x + \epsilon_i) \nabla_{\epsilon_i} - \log \mu(\epsilon_i)] \quad (7)$$

$\epsilon_1, \dots, \epsilon_s \sim \mu.$

Corollary 2.2 (Differentiable Density Smoothing for Vector-valued Functions). *We can extend Lemma 2.1 to vector-valued functions $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, allowing to compute the Jacobian matrix $\mathbf{J}_{f_\epsilon} \in \mathbb{R}^{m \times n}$ as*

$$\mathbf{J}_{f_\epsilon}(x) = \mathbb{E}_{\epsilon \sim \mu} [f(x + \epsilon) \cdot (\nabla_\epsilon - \log \mu(\epsilon))^\top]. \quad (8)$$

We remark that prior work (e.g., (Abernethy et al., 2016)) limits μ to be a differentiable density with full support on $\mathbb{R}$, typically of exponential family, whereas we generalize it to any absolutely continuous density, and include additional generalizations. This has important implications for distributions such as Cauchy, Laplace, and Triangular, which we show to have considerable practical relevance.

Lemma 2.3 (Requirement of Continuity of μ). *If $\mu(\epsilon)$ is absolutely continuous (and not necessarily differentiable), then f_ϵ is continuous and differentiable everywhere.*

$$\nabla_x f_\epsilon(x) = \mathbb{E}_{\epsilon \sim \mu} [f(x + \epsilon) \cdot \mathbf{1}_{\epsilon \notin \Omega} \cdot \nabla_\epsilon - \log \mu(\epsilon)]. \quad (9)$$

Ω is the zero-measure set of points with undefined gradient. We provide the proof in Appendix A.1.

Lemma 2.3 has important implications. In particular, it enables smoothing with non-differentiable density distributions such as the Laplace distribution, the triangular distribution, and the Wigner Semicircle distribution (Wigner, 1955; 1958) while maintaining differentiability of f_ϵ .

Remark 2.4 (Requirement of Continuity of μ). However, it is crucial to mention that, for stochastic smoothing (Lemmata 2.1, 2.3, Corollary 2.2), μ has to be continuous. E.g., the uniform distribution is *not* a valid choice because it does not have a continuous density on $\mathbb{R}$. ($\mathcal{U}(a, b)$ has discontinuities at a, b where it jumps between 0 and $1/(b - a)$.) With other formulations, e.g., (Berahas et al., 2022; Flaxman et al., 2004), it is possible to perform smoothing with a uniform distribution over a ball; however, if f is discontinuous, uniform smoothing may not lead to a differentiable function. Continuity is necessary but not sufficient, and absolutely continuous is sufficient; however, the difference to continuity consists only in non-practical and adversarial examples, e.g., the Cantor or Weierstrass functions.

Remark 2.5 (Gaussian Smoothing). A popular special case of differentiable stochastic smoothing is smoothing with a Gaussian distribution $\mu_{\mathcal{N}} = N(\mathbf{0}_n, \mathbf{I}_n)$. Here, due to the nature of the probability density function of a Gaussian, $\nabla_{\epsilon} - \log \mu_{\mathcal{N}}(\epsilon) = \epsilon$. Further, when $\mu_{\mathcal{N}_{\sigma}} = N(\mathbf{0}_n, \sigma^2 \mathbf{I}_n)$, then $\nabla_{\epsilon} - \log \mu_{\mathcal{N}_{\sigma}}(\epsilon) = \epsilon/\sigma$. We emphasize that this equality *only* holds for the Gaussian distribution.

Equipped with the core idea behind stochastic smoothing, we can differentiate any function f via perturbation with a probability distribution with (absolutely) continuous density. Typically, probability distributions that we consider for smoothing are parameterized via a scale parameter, viz., the standard deviation σ in a Gaussian distribution or the scale γ in a Cauchy distribution. Extending the formalism above, we may be interested in differentiating with respect to the scale parameter γ of our distribution μ . This becomes especially attractive when optimizing the scale and, thereby, degree of relaxation of our probability distribution. While our formalism allows reparameterization to express γ within μ , we can also explicitly write it as

$$\begin{aligned} \nabla_x f_{\gamma\epsilon}(x) &= \nabla_x \mathbb{E}_{\epsilon \sim \mu} [f(x + \gamma \cdot \epsilon)] \\ &= \mathbb{E}_{\epsilon \sim \mu} [f(x + \gamma \cdot \epsilon) \cdot (\nabla_{\epsilon} - \log \mu(\epsilon)) / \gamma]. \end{aligned} \quad (10)$$

Now, we can differentiate wrt. γ , i.e., compute $\nabla_{\gamma} f_{\gamma\epsilon}(x)$.

Lemma 2.6 (Differentiation wrt. γ). *Extending Lemma 2.1, Corollary 2.2, and Lemma 2.3, we have*

$$\begin{aligned} \nabla_{\gamma} f_{\gamma\epsilon}(x) &= \nabla_{\gamma} \mathbb{E}_{\epsilon \sim \mu} [f(x + \gamma \cdot \epsilon)] \\ &= \mathbb{E}_{\epsilon \sim \mu} [f(x + \gamma \cdot \epsilon) \cdot (-1 + (\nabla_{\epsilon} - \log \mu(\epsilon))^{\top} \cdot \epsilon) / \gamma]. \end{aligned} \quad (11)$$

The proof is deferred to Appendix A.2.

We can extend γ for multivariate distributions to a scale matrix $\Sigma/\mathbf{L}$ (e.g., a covariance matrix). This enables optimization for finding the optimal scale matrix $\mathbf{L}$, and further not only isotropic distributions, but instead also, e.g., multivariate Gaussians with correlations between dimensions.

Theorem 2.7 (Multivariate Smoothing with Cov. Matrix). *We have a function $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$. We assume ϵ is drawn from a multivariate distribution with absolutely continuous density in $\mathbb{R}^n$. Let $\mathbf{L} \in \mathbb{R}^{n \times n}$ be an invertible scale*

matrix (e.g., for a covariance matrix Σ , $\mathbf{L}$ is based on its Cholesky decomp. $\mathbf{L}\mathbf{L}^{\top} = \Sigma$). We define $f_{\mathbf{L}\epsilon}(x) = \mathbb{E}_{\epsilon \sim \mu} [f(x + \mathbf{L} \cdot \epsilon)]$. Then, $\partial f_{\mathbf{L}\epsilon}(x) / \partial x \in \mathbb{R}^{m \times n}$ and $\partial f_{\mathbf{L}\epsilon}(x) / \partial \mathbf{L} \in \mathbb{R}^{m \times n \times n}$ can be computed as

$$\nabla_x (f_{\mathbf{L}\epsilon}(x))_i = \quad (12)$$

$$\mathbb{E}_{\epsilon \sim \mu} [f(x + \mathbf{L} \cdot \epsilon)_i \cdot \mathbf{L}^{-1} \cdot (\nabla_{\epsilon} - \log \mu(\epsilon))],$$

$$\nabla_{\mathbf{L}} (f_{\mathbf{L}\epsilon}(x))_i = \quad (13)$$

$$\mathbb{E}_{\epsilon \sim \mu} [f(x + \mathbf{L} \cdot \epsilon)_i \cdot \mathbf{L}^{-\top} \cdot (-1 + (\nabla_{\epsilon} - \log \mu(\epsilon)) \cdot \epsilon^{\top})].$$

Above, the indicator (from (9)) is omitted for a simplified exposition. Proofs are deferred to Apx. A.3.

In Appendix C, we additionally extend stochastic smoothing to differentiating the expected k -sample median, show that it is differentiable, and provide an unbiased gradient estimator in Lemma C.2. In Appendix D and Theorem D.1 we show how to differentiate the covariance matrices of outputs.

2.1. Distribution Examples

After covering the underlying theory of generalized stochastic smoothing, in this section, we provide examples of specific distributions that our theory applies to. We illustrate the distributions in Table 1. Before delving into individual choices, we provide a clarification for multivariate densities $\mu: \mathbb{R}^n \rightarrow \mathbb{R}_{\geq 0}$: We consider the n -dimensional multivariate form of a distribution as the concatenation of n independent univariate distributions. Thus, for μ_1 as the univariate formulation of the density, we have the proportionality $\mu(\epsilon) \propto \prod_{i=1}^n \mu_1(\epsilon_i)$. We remark that the distribution by which we smooth ($\mathbf{L}\epsilon$) is not an isotropic (per-dimension independent) distribution. Instead, through transformation by the scale matrix $\mathbf{L}$, e.g., in the case of the Gaussian distribution, $\mathbf{L}\epsilon$ covers the entire space of multivariate Gaussian distributions with arbitrary covariance matrices.

Beyond the **Gaussian** distribution, the **logistic** distribution offers heavier tails, and the **Gumbel** distribution provides max-stability, which can be important for specific tasks. The **Cauchy** distribution (Ferguson, 1962), with its undefined mean and infinite variance, also has important implications in smoothing: e.g., it has been shown to provide monotonicity in differentiable sorting networks (Petersen et al., 2022a). While prior art (Lidec et al., 2021) heuristically utilized the Cauchy distribution for stochastic smoothing of argmax, this had been, thus far, without a general formal justification.

In this work, for the first time, we consider Laplace and triangular distributions. First, the **Laplace** distribution, as the symmetric extension of the exponential distribution, does not lie in the space of exponential family distributions, and is not differentiable at 0. Via Lemma 2.3, we show that stochastic smoothing can still be applied and is exactly correct despite non-differentiability of the distribution. A benefit of the Laplace distribution is that all samples contribute

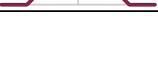
Distribution	Density / PDF $\mu(\epsilon)$	$\nabla_{\epsilon} - \log \mu(\epsilon)$	$\mu(\epsilon) \cdot \nabla_{\epsilon} - \log \mu(\epsilon)$
Gaussian	 $\frac{1}{\sqrt{2\pi}} \exp(-1/2 \cdot \epsilon^2)$	ϵ	
Logistic	 $\frac{\exp(-\epsilon)}{(1 + \exp(-\epsilon))^2}$	$\tanh(\epsilon/2)$	
Gumbel	 $\exp(-\epsilon - \exp(-\epsilon))$	$1 - \exp(-\epsilon)$	
Cauchy	 $\frac{1}{\pi \cdot (1 + \epsilon^2)}$	$\frac{2 \cdot \epsilon}{1 + \epsilon^2}$	
Laplace	 $1/2 \cdot \exp(- \epsilon)$	$\text{sign}(\epsilon)$	
Triangular	 $\max(0, 1 - \epsilon)$	$\frac{\text{sign}(\epsilon)}{1 - \epsilon } (*)$	

Table 1. Probability distributions considered for generalized stochastic smoothing. Displayed is (from left to right) the density of the distribution $\mu(\epsilon)$ (plot + equation), the derivative of the NLL (equation), and the product between the density and the derivative of the NLL (plot). The latter plot corresponds to the kernel that a function f to be smoothed is effectively convolved by to estimate the gradient.

(*): applies to $\epsilon \in (-1, 1) \setminus \{0\}$, otherwise 0 or undefined.

equally to the gradient computation ($|\nabla_{\epsilon} - \log \mu(\epsilon)| = 1$), reducing variance. Second, with the **triangular** distribution, we illustrate, for the first time, that stochastic smoothing can be performed even with a non-differentiable distribution with compact support ($[-1, 1]$). This is crucial if the domain of f has to be limited to a compact set, or in applications where smoothing beyond a limited distance to the original point is not meaningful. A hypothetical application for this is differentiating a physical motor controlled robot in reinforcement learning where we may not want to support an infinite range for safety considerations.

2.2. Variance Reduction

Given an unbiased estimator of the gradient, e.g., in its simplest form (2), we desire reducing its variance, or, in other words, improve the quality of the gradient estimate for a given number of samples. For this, we consider 3 orthogonal perspectives of variance reduction: covariates, antithetic samples, and (randomized) quasi-Monte Carlo.

To illustratively derive the first two variance reductions, let us consider the case of smoothing a constant function $f(x) = v$ for some large constant $v \gg 0$. Naturally, $f_{\epsilon}(x) = v$ and $\nabla_x f_{\epsilon}(x) = 0$. However, for a finite number of samples s , our empirical estimate (e.g., (7)) will differ from 0 almost surely. As the gradient of $f_{\epsilon}(x) - c$ wrt. x does not depend on c , we have $\nabla_x f_{\epsilon}(x) = \nabla_x (f_{\epsilon}(x) - c)$.

If we choose $c = v$, the variance of the gradient estimator is reduced to 0. For general and non-constant f , we can estimate the optimal choice of c via $c = f(x)$ or via the leave-one-out estimator (Quenouille, 1956; Tukey, 1958) of $f_{\epsilon}(x)$. In the fields of stochastic smoothing of optimizers and reinforcement learning this is known as the *method of covariates*. $f(x)$ and LOO were previously considered for smoothing, e.g., in (Lidec et al., 2021) and (Mimori & Hamada, 2023), respectively. We illustrate the effects of both choices of covariates in Figure 1.

From an orthogonal perspective, we observe that $\mathbb{E}_{\epsilon \sim \mu}[\nabla_{\epsilon} - \log \mu(\epsilon)] = 0$, which follows, for example, from

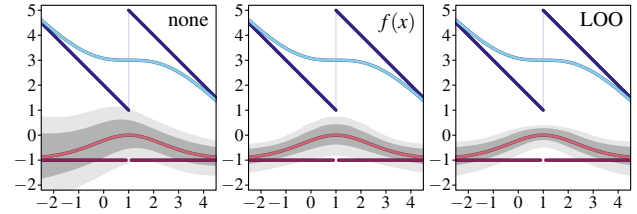


Figure 1. Comparison of covariates: a non-differentiable function (dark blue) is smoothed with a logistic distribution (light blue). The original gradient (dark red) is not everywhere defined, and does not meaningfully represent the gradient of the smoothed function. The gradient of the smoothed function is shown in pink. Grey illustrates the variance of a gradient estimate with 5 samples via the [25%, 75%] (dark grey) and [10%, 90%] (light grey) percentiles. Using $f(x)$ as a covariate, instead of using *none* reduces the gradient variance, in particular whenever $f(x)$ is large. Leave-one-out (LOO) further improves over $f(x)$ at discontinuities of the original function f (i.e., at $x=1$), but has slightly higher variance than $f(x)$ where f is continuous and has large values (i.e., at $x=-2$).

$\nabla_x f_{\epsilon}(x) = 0 = \mathbb{E}_{\epsilon \sim \mu}[v \cdot \nabla_{\epsilon} - \log \mu(\epsilon)]$. For symmetric distributions, we can guarantee an empirical estimate to be 0 by always using pairs of *antithetic samples* (Hammersley & Morton, 1956), i.e., complementary ϵ s. Using $\epsilon' = -\epsilon$, we have $\nabla_{\epsilon} \log \mu(\epsilon) + \nabla_{\epsilon'} \log \mu(\epsilon') = 0$. This is illustrated in Figure 2 (2). In our experiments, we observe antithetic sampling to perform poorly in comparison to other methods.

A third perspective considers that points sampled with standard Monte Carlo (MC) methods (see Fig. 2 (1)), due to the random nature of the sampling, often form (accidental) clumps while other areas are void of samples. To counteract this, quasi-Monte Carlo (QMC) methods (Metropolis & Ulam, 1949) spread out sampled points as evenly as possible by foregoing randomness and choosing points from a regular grid, e.g., a simple Cartesian grid, taking the grid cell centers as samples (see Fig. 2 (3)). Via the inverse CDF of the respective distribution, the points can be mapped from the unit hypercube to samples from a respective distribution. However, discarding randomness makes the sampling process deterministic, limits the dispersion introduced by the smoothing distribution to concrete points, and hence

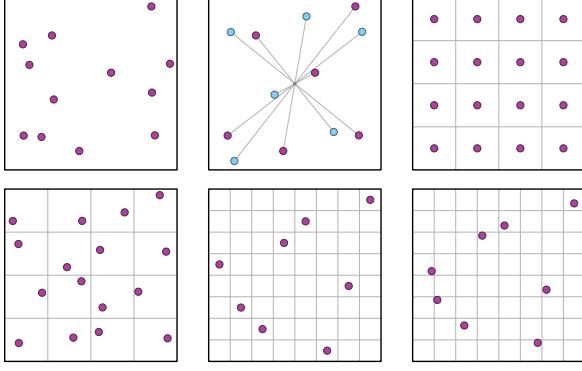


Figure 2. Sampling strategies. Top, left to right: Monte-Carlo (MC), Antithetic Monte-Carlo, Cartesian Quasi-Monte-Carlo (QMC); bottom, left to right: Cartesian Randomized-Quasi-Monte-Carlo (RQMC), Latin-Hypercube Sampled QMC and RQMC. Samples can be transformed via the inverse CDF of a respective distribution.

makes the estimator biased. Randomized quasi-Monte Carlo (RQMC) (Niederreiter, 1978) methods overcome this difficulty by reintroducing some randomness. Like QMC, RQMC uses a grid to subdivide $[0, 1]^n$ into cells, but then samples a point from each cell (see Fig. 2 (4)) instead of taking the grid cell center. While regular MC sampling leads to variances of $\mathcal{O}(1/\sqrt{s})$, RQMC reduces them to $\mathcal{O}(1/s^{1+2/n})$ for a number s of samples and an input dimension of n (L’Ecuyer, 2018). For large n , we still have a rate of at least $\mathcal{O}(1/s) \supset \mathcal{O}(1/s^{1+2/n})$, which constitutes a substantial improvement over the regular reduction in $\mathcal{O}(1/\sqrt{s})$. However, the default (i.e., Cartesian) QMC and RQMC methods require numbers of samples $s = k^n$ for $k \in \mathbb{N}_+$, which can become infeasible for large input dimensionalities. Therefore, we also consider Latin-Hypercube Sampling (LHS) (McKay et al., 1979), which uses a subset of grid cells such that each interval in every dimension is covered exactly once (see Fig. 2 (5+6)). We remark that, to our knowledge, we are the first to consider QMC and RQMC sampling strategies for gradient estimation.

2.3. Smoothing of the Algorithm vs. the Objective

In algorithmic supervision settings (Petersen et al., 2021b), we can write our training objective as $\ell(h(y))$ where y is the output of a neural network model, h is the algorithm, and the scalar function ℓ is the training objective (loss function) applied to the output of the algorithms. In such cases, we can distinguish between smoothing the algorithms (i.e., $f = h$) and smoothing the loss (i.e., $f = \ell \circ h$).

When *smoothing the algorithm*, we compute the value and derivative of $\ell(\mathbb{E}_\epsilon[h(y + \epsilon)])$. This requires our loss ℓ to be differentiable and capable of receiving relaxed inputs. (E.g., if the output of h is binary, then ℓ has to be able to operate on real-valued inputs from $(0, 1)$.) In this case, the derivative of $\mathbb{E}_\epsilon[h(y + \epsilon)]$ is a Jacobian matrix (see Corollary 2.2).

When *smoothing the objective / loss function*, we compute

the value and derivative of $\mathbb{E}_\epsilon[\ell(h(y + \epsilon))]$. Here, the objective / loss ℓ does not need to be differentiable and can be limited to operate on discrete outputs of the algorithm h . Here, the derivative of $\mathbb{E}_\epsilon[\ell(h(y + \epsilon))]$ is a gradient.

Whether smoothing the algorithm or the objective is better depends on different factors including the problem setting and algorithm, the availability of a real-variate and real-valued ℓ , and how many samples can be afforded. In practice, we observe that, whenever we can afford large numbers of samples, smoothing of the algorithm performs better.

3. Related Work

In the theoretical literature of gradient-free optimization, stochastic smoothing has been extensively studied (Glasserman, 1990; Yousefian et al., 2010; Duchi et al., 2012; Abernethy et al., 2016). Our work extends existing results, generalizing the set of allowed distributions, considering vector-valued functions, anisotropic scale matrices, enabling k -sample median differentiation, and a characterization of finite definedness of expectations and their gradients based on the relationship between characteristics of the density and smoothed functions.

From a more applied perspective, stochastic smoothing has been applied for relaxing convex optimization problems (Berthet et al., 2020; Lidec et al., 2021; Stewart et al., 2023). In particular, convex optimization formulations of argmax (Berthet et al., 2020; Lidec et al., 2021), the shortest-path problem (Berthet et al., 2020), and the clustering problem (Stewart et al., 2023) have been considered. We remark that the perspective of smoothing any function or algorithm f , as in this work, differs from the perspective of perturbed optimizers. In particular, optimizers are a special case of the functions we consider.

While we consider smoothing functions with real-valued inputs, there is also a rich literature of differentiating stochastic discrete programs (Krieken et al., 2021; Arya et al., 2022; Kagan & Heinrich, 2023). These works typically use the inherent stochasticity from discrete random variables in programs and explicitly model the internals of the programs. We consider real-variate black-box functions and smooth them with added input noise.

In the literature of reinforcement learning, a special case or analogous idea to stochastic smoothing can be found in the REINFORCE formulation where the (scalar) score function is smoothed via a policy (Williams, 1992; Schmidhuber, 1990; Abernethy et al., 2016; Sutton & Barto, 2018). Compared to the literature, we enable new distributions and respective characterizations of requirements for the score functions. We hope our results will pave their way into future RL research directions as they are also applicable to RL without major modification. Individual related work

for respective applications is discussed in the respective experiment sections.

4. Experiments

For the experiments, we consider 4 experimental domains: sorting & ranking, graph algorithms, 3D mesh rendering, and cryo-electron tomography (cryoET) simulations. The primary objective of the empirical evaluations is to compare different distributions as well as different variance reduction techniques. We begin our evaluations by measuring the variance of the gradient estimators, then continue with optimizations and using the differentiable relaxations in deep learning tasks. We remark that, in each experiment, f does not have any non-zero gradients, and thus using first-order or path-wise gradients or gradient estimators is not possible.

4.1. Variance of Gradient Estimators

We evaluate the gradient variances for different variance reduction techniques in Figures 3 and 10. For differentiable sorting and ranking, we smooth the (hard) permutation matrix that sorts an input vector ($f : \mathbb{R}^n \rightarrow \{0, 1\}^{n \times n}$). For diff. shortest-paths, we smooth the function that maps from a 2D cost-map to a binary encoding of the shortest-path under 8-neighborhood ($f : \mathbb{R}^{n \times n} \rightarrow \{0, 1\}^{n \times n}$). Both functions are not only non-differentiable, but also have no non-zero gradients anywhere. For each distribution, we compare all combinations of the 3 variance reduction techniques.

On the axis of sampling strategy, we can observe that, whenever available, Cartesian RQMC delivers the lowest variance. The only exception is the triangular distribution, where latin QMC provides the lowest uncentered gradient variance (despite being a biased estimator) because of large contributions to the gradient for samples close to -1 and 1 . Between latin QMC and RQMC, we can observe that their variance is equal except for the high-dimension cases of the Cauchy distribution and a few cases of the Gumbel distribution, where QMC is of lower variance. However, due to the bias in QMC, RQMC would typically still be preferable over QMC. We do not consider Cartesian QMC due to its substantially greater bias. In heuristic conclusion, $\text{RQMC}(c.) \succ \text{RQMC}(l.) \succeq \text{QMC}(l.) \succ \text{MC}$.

On the axis of using antithetic sampling (left vs. right in each subplot), we observe that it consistently performs worse than the regular counterpart, except for vanilla MC without a covariate. The reason for this is that antithetic sampling does not lead to a good sample-utilization trade-off once we consider quasi Monte-Carlo strategies. Thus, in the following, we consider antithetic only for MC.

On the axis of the covariate, we observe that LOO consistently provides the lowest gradient variances. This aligns with intuition from Figure 1 where LOO provides the lowest variance at discontinuities (in this subsection, f is discontinuous or constant everywhere).

Comparing the case of no covariate and $f(x)$, which is better depends on the individual setting, which makes sense considering the binary outputs of the algorithms. $f(x)$ performs well for functions that attain large values while having fewer discontinuities.

In conclusion, the best setting is Cartesian RQMC with the LOO covariate and without antithetic sampling whenever available (only for $s = k^n$ samples for $k \in \mathbb{N}$). The next best choice is typically RQMC with Latin hypercube sampling. In Appendix E, we provide additional analysis, showing that, depending on the distribution, RQMC reduces variances by factors of up to 2 585 in the case of 100 samples, and even larger factors for more samples.

4.2. Differentiable Sorting & Ranking

After investigating the choices of variance reduction techniques wrt. the variance alone, in this section, we explore the utility of stochastic smoothing on the 4-digit MNIST sorting benchmark (Grover et al., 2019). Here, at each step, a set of $n=5$ 4-digit MNIST images (such as ) is presented to a CNN, which predicts the displayed scalar value for each of the n images independently. For training the model, no absolute information about the displayed value is provided, and only the ordering or ranking of the n images according to their ground truth value is supervised. The goal is to learn an order-preserving CNN, and the evaluation metric is the fraction of correctly inferred orders from the CNN (exact match accuracy). Training the CNN requires a differentiable ranking operator (that maps from a vector to a differentiable permutation matrix) for the ranking loss. Previous work has considered NeuralSort (Grover et al., 2019), SoftSort (Prillo & Eisenschlos, 2020), casting sorting as a regularized OT problem (Cuturi et al., 2019), and differentiable sorting networks (DSNs) (Petersen et al., 2021a; 2022a). The state-of-the-art is monotonic DSN (Petersen et al., 2022a), which utilizes a relaxation based on Cauchy distributions to provide monotonic differentiable sorting, which has strong theoretical and empirical advantages.

In Figure 4, we evaluate the performance of generalized stochastic smoothing with different distributions and different numbers of samples for each variance reduction method. We observe that, while the Cauchy distribution performs poorly for small numbers of samples, it performs best for large numbers. This makes sense as the Cauchy distribution has infinite variance and, for DSNs, provides monotonicity. We remark that large numbers of samples can easily be afforded in many applications (when comparing the high cost of neural networks to the vanishing cost of sorting/ranking within a loss function). Nevertheless, for 32 768 samples, the sorting operation starts to become the bottleneck. The Laplace distribution is the best choice for smaller numbers of samples, which aligns with the characterization of it having the lowest variance because all

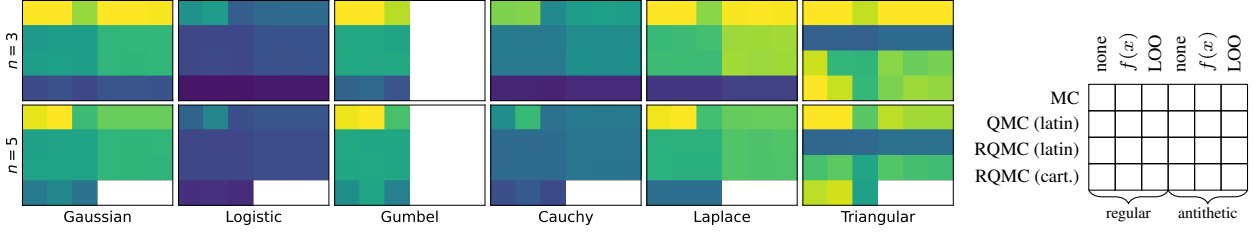


Figure 3. Average L_2 norms between ground truth (oracle) and estimated gradient for different numbers of elements to sort and rank n , and different distributions. Each plot compares different variance reduction strategies, see legend on the right. *Darker is better* (smaller values). Colors are only comparable within each subplot because the magnitudes of gradients differ between distributions and dimensionalities. We use 1 024 samples, except for Cartesian and $n = 3$ where we use $10^3 = 1\,000$ samples. An extension with $n \in \{7, 10\}$ can be found in Figure 11 in the appendix. Absolute values are reported in Table 8.

Table 2. Sorting benchmark results ($n = 5$), avg. over 12 seeds. ‘best (cv)’ refers to the best sampling strategy, as determined via cross-validation (thus, there is no bias from the selection of the strategy). Table 7 includes additional numbers of samples and standard deviations. Baselines are NeuralSort (Grover et al., 2019), SoftSort (Prillo & Eisenschlos, 2020), Logistic DSNs (Petersen et al., 2021a), Cauchy and Error-opt. DSNs (Petersen et al., 2022a), and OT Sort (Cuturi et al., 2019), averaged over 5 seeds.

Baselines	Neu.S.	Soft.S.	L. DSN	C. DSN	E. DSN	OT. S.
—	71.3	70.7	77.2	84.9	85.0	81.1
Sampling #s	Gauss.	Logis.	Gumbel	Cauchy	Laplace	Trian.
vanilla 256	82.3	82.8	79.2	68.1	82.6	81.3
best (cv) 256	83.1	82.7	81.6	55.6	83.7	82.7
vanilla 1k	81.3	83.7	82.0	68.5	80.6	82.8
best (cv) 1k	83.9	84.0	84.2	73.0	84.3	82.4
vanilla 32k	84.2	84.1	84.5	84.9	84.4	83.4
best (cv) 32k	84.4	84.4	84.8	85.1	84.4	84.0

samples contribute equally to the gradient. Wrt. variance reduction, we continue to observe that vanilla MC performs worst. RQMC performs best, except for Triangular, where QMC is best. For the Gumbel distribution, we observe reduced performance for Latin hypercube sampling. Generally, we observe that $f(x)$ is the worst choice of covariate, but the effect lies within standard deviations. In Table 2, we provide a numerical comparison to other differentiable sorting approaches. We observe that all choices of distributions improve over all baselines except for the monotonic DSNs, even at smaller numbers of samples (i.e., without measurable impact on training speed). Finally, the Cauchy distribution leads to a small improvement over the SOTA, without requiring a manually designed differentiable sorting algorithm; however, only at the computational cost of 32 768 samples.

4.3. Differentiable Shortest-Paths

The Warcraft shortest-path benchmark (Vlastelica et al., 2020) is the established benchmark for differentiable shortest-path algorithms (e.g., (Vlastelica et al., 2020; Berthet et al., 2020; Petersen et al., 2021b)). Here, a War-

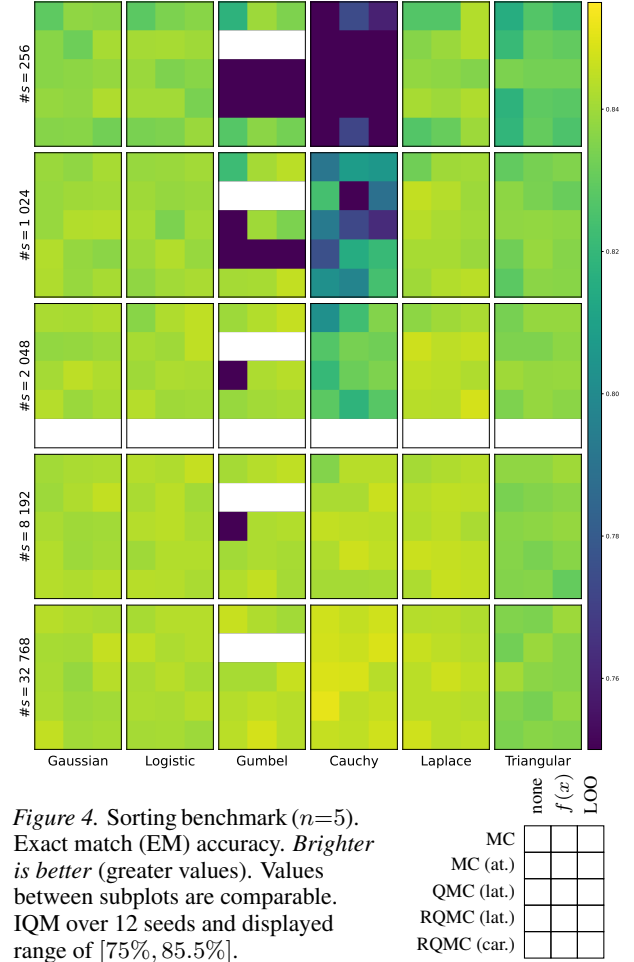


Figure 4. Sorting benchmark ($n=5$). Exact match (EM) accuracy. *Brighter is better* (greater values). Values between subplots are comparable. IQM over 12 seeds and displayed range of [75%, 85.5%].

craft pixel map is provided, a CNN predicts a 12×12 cost matrix, a differentiable algorithm computes the shortest path, and the supervision is only the ground truth shortest path. Berthet et al. (Berthet et al., 2020) considered stochastic smoothing with Fenchel-Young (FY) losses, which improves sample efficiency for small numbers of samples. However, the FY loss does not improve for larger numbers of samples (e.g., Tab. 7.5 in (Petersen, 2022)).

As computing the shortest-path is computationally efficient and parallelizable (our implementation is $\approx 5\,000 \times$

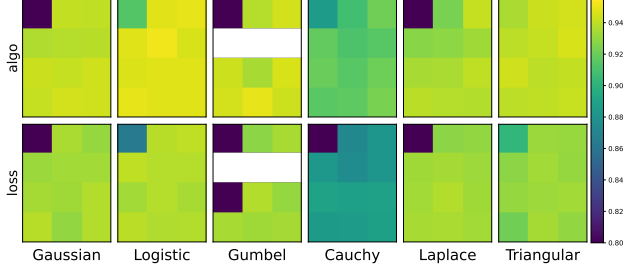


Figure 5. Warcraft shortest-path experiment with 1000 samples. *Brighter is better* (larger values).

Values between subplots are comparable.

Exact match accuracy avg. over 5 seeds and displayed range [80%, 95.5%].

Additional settings in Figs. 13 and 14.

	none	$f(x)$	LOO
MC			
MC (at.)			
QMC (lat.)			
RQMC (lat.)			

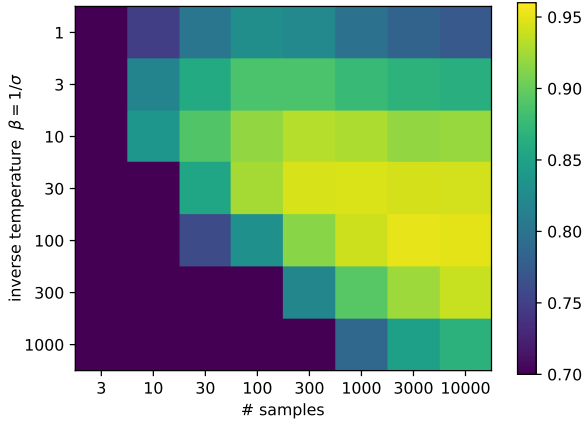


Figure 6. Warcraft shortest-path experiment using Gaussian smoothing of the algorithm (RQMC with latin hypercube-sampling and LOO covariate). Comparing the effects between the inverse temperature β and the number of samples. We observe that with growing numbers of samples, the optimal inverse temperature increases, i.e., the optimal standard deviation for the Gaussian noise decreases. Averaged over 5 seeds.

faster than the Dijkstra implementation used in previous work (Berthet et al., 2020; Vlastelica et al., 2020)), we can afford substantially larger numbers of samples, improving the quality of gradient estimation. In Figure 5, we compare the performance of different smoothing strategies. The logistic distribution performs best, and smoothing of the algorithm (top) performs better than smoothing of the loss (bottom). Variance reduction via sampling strategies (antithetic, QMC, or RQMC) improves performance, and the best covariate is LOO. For reference, the FY loss (Berthet et al., 2020) leads to an accuracy of 80.6%, regardless of the number of samples. Generalized stochastic smoothing (GSS) consistently achieves 90%+ using 100 samples (see Fig. 13 right). Using 10000 samples, and variance reduction, we achieve 96.6% in the best setting (Fig. 14) compared to the SOTA of 95.8% (Petersen et al., 2021b). In Fig. 6, we illustrate that smaller standard deviations (larger β) are better for more samples.

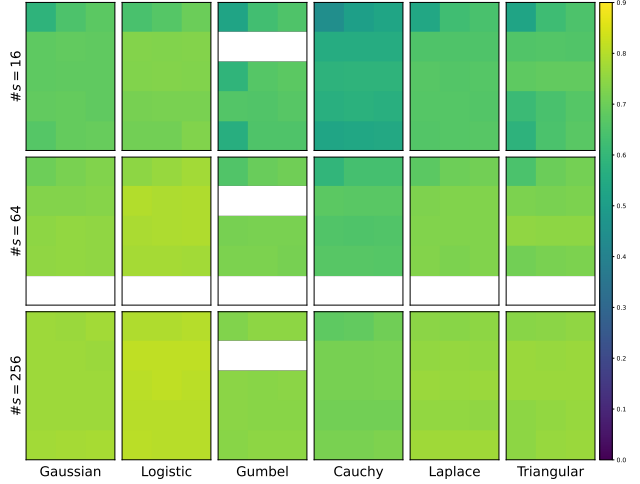


Figure 7. Utah teapot camera pose optimization. The

metric is fraction of camera poses recovered; the initialization angle errors are uniformly distributed in $[15^\circ, 75^\circ]$. *Brighter is better*.

Avg. over 768 seeds. The displayed range is [0%, 90%].

	none	$f(x)$	LOO
MC			
MC (at.)			
QMC (lat.)			
RQMC (lat.)			
RQMC (car.)			

4.4. Differentiable Rendering

For differentiable rendering (Loper & Black, 2014; Kato et al., 2018; Petersen et al., 2019; Kato et al., 2020; Lidec et al., 2021; Petersen et al., 2022c;b; Fischer & Ritschel, 2023a;b), we smooth a non-differentiable hard renderer via sampling. This differs from DRPO (Lidec et al., 2021), which uses stochastic smoothing to relax the Heaviside and Argmax functions within an already differentiable renderer. Instead, we consider the renderer as a black-box function. This has the advantage of noise parameterized in the coordinate space rather than the image space.

We benchmark stochastic smoothing for rendering by optimizing the camera-pose (4-DoF) for a Utah teapot, an experiment inspired by (Lidec et al., 2021; Petersen et al., 2022b). We illustrate the results in Figure 7. Here, the logistic distribution performs best, and QMC/RQMC as well as LOO lead to the largest improvements. While Fig. 7 shows smoothing the rendering algorithm, Fig. 12 performs smoothing of the training objective / loss. Smoothing the alg. is better because the loss (MSE), while well-defined on discrete renderings, is less meaningful on disc. renderings.

4.5. Differentiable Cryo-Electron Tomography

Transmission Electron Microscopy (TEM) transmits electron beams through thin specimens to form images (Williams & Carter, 1996). Due to the small electron beam wavelength, TEM leads to higher resolutions of up to single columns of atoms. Obtaining high resolution images from TEM involves adjustments of various experimental parameters. We apply smoothing to a realistic black-box

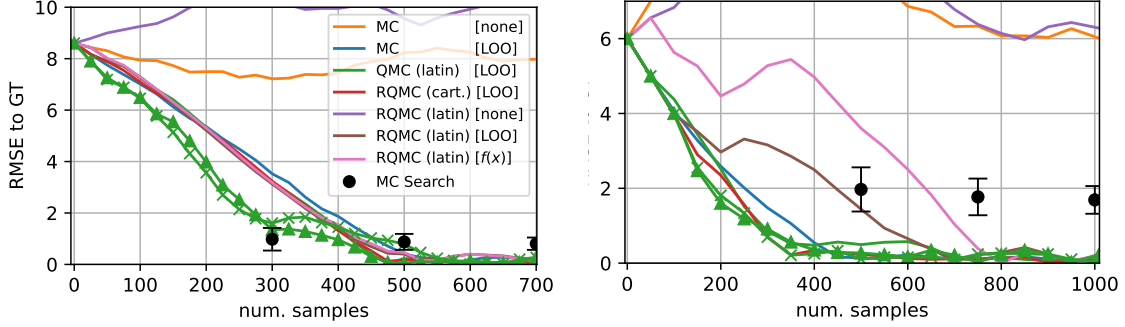


Figure 8. RMSE to Ground Truth parameters for the 2-parameter (left) and 4-parameter experiment (right). We optimize the L_2 loss between generated and GT images using loss smoothing. No marker lines correspond to Gaussian, $\times$ to Laplace and $\triangle$ to Triangular distributions. Laplace and Triangular perform best; LOO leads to the largest improvements. Add. results are in Figure 15.

TEM simulator (Rullgård et al., 2011), optimizing sets of parameters to approximate reference Tobacco Mosaic Virus (TMV) (Sachse et al., 2007) micrographs. In Figure 8, we perform two experiments: a 2-parameter study optimizing the microscope acceleration voltage and x -position of the specimen, and a 4-parameter study with additional parameters of the particle’s y -position and the primary lens focal length. The micrograph image sizes are 400×400 pixels, and accordingly we use smoothing of the loss.

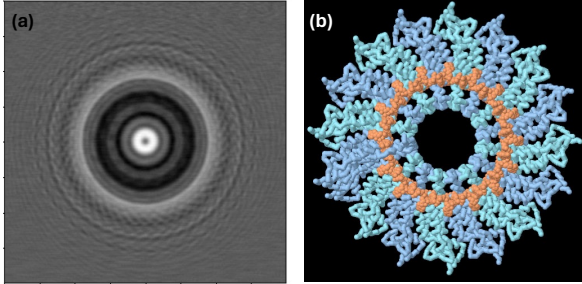


Figure 9. (a) Simulated Transmission Electron micrograph, (b) TMV structure with RNA (orange) and protein stacks (blue).

Summary of Experimental Results Generally, we observe that QMC and RQMC perform best, whereas antithetic sampling performs rather poorly. In low-dimensional problems, it is advisable to use RQMC (cartesian), and in higher dimensional problems (R)QMC (latin), still works well. As for the covariate, LOO typically performs best; however, the choice of sampling strategy (QMC/RQMC) is more important than choosing the covariate. In sorting and ranking, the Cauchy distribution performs best for large numbers of samples and for smaller numbers of samples, the Laplace distribution performs best. In the shortest-path case, the logistic distribution performs best, and Gaussian closely follows. Here, we also observe that with larger numbers of samples, the optimal standard deviation decreases. For differentiable rendering, the logistic distribution performs best.

Limitations A limitation of zeroth-order gradient estimators is that they are generally only competitive if the first-order gradients do not exist (see (Suh et al., 2022) for discussions of exceptions). In this vein, in order to be competitive with custom designed continuous relaxations like a differentiable renderer, we may need a very large number of samples, which could become prohibitive for expensive functions f . The optimal choice of distribution depends on the function to be smoothed, no singular distribution is optimal for all f ; however, if one wants to limit the distribution to a single choice, we recommend the logistic or Laplace distribution, as, with their simple exponential convergence, they give a good middle ground between heavy-tailed and light-tailed distributions. Finally, variance reduction techniques like QMC/RQMC are not immediately applicable in single sample settings, and the variance reduction techniques in this paper build on evaluating f many times.

5. Conclusion

In this work, we derived stochastic smoothing with reduced assumptions and outlined a general framework for relaxation and gradient estimation of non-differentiable black-box functions. This enables an increased set of distributions for stochastic smoothing, e.g., enabling smoothing with the triangular distribution while maintaining full differentiability of f_ϵ . We investigated variance reduction for stochastic smoothing-based gradient estimation from 3 orthogonal perspectives, finding that RQMC and LOO are generally the best methods, whereas the popular antithetic sampling method performs rather poorly. Moreover, enabled by supporting vector-valued functions, we disentangled the algorithm and objective, thus smoothing f while analytically backpropagating through the loss ℓ , improving gradient estimation. We applied stochastic smoothing to differentiable sorting and ranking, diff. shortest-paths on graphs, diff. rendering for pose estimation and diff. cryo-ET simulations. We hope that our work inspires the community to develop their own stochastic relaxations for differentiating non-differentiable algorithms, operators, and simulators.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Abernethy, J., Lee, C., and Tewari, A. Perturbation techniques in online learning and optimization. *Perturbations, Optimization, and Statistics*, 2016.
- Arvo, J. and Kirk, D. Fast ray tracing by ray classification. *ACM Siggraph Computer Graphics*, 21(4):55–64, 1987.
- Arya, G., Schauer, M., Schäfer, F., and Rackauckas, C. Automatic differentiation of programs with discrete randomness. *Advances in Neural Information Processing Systems*, 35:10435–10447, 2022.
- Berahas, A. S., Cao, L., Choromanski, K., and Scheinberg, K. A theoretical and empirical comparison of gradient approximations in derivative-free optimization. *Foundations of Computational Mathematics*, 22(2):507–560, 2022.
- Berthet, Q., Blondel, M., Teboul, O., Cuturi, M., Vert, J.-P., and Bach, F. Learning with Differentiable Perturbed Optimizers. In *Proc. Neural Information Processing Systems (NeurIPS)*, 2020.
- Blondel, M., Teboul, O., Berthet, Q., and Djolonga, J. Fast Differentiable Sorting and Ranking. In *Proc. International Conference on Machine Learning (ICML)*, 2020.
- Chu, J. T. and Hotelling, H. The moments of the sample median. *The Annals of Mathematical Statistics*, pp. 593–606, 1955.
- Cuturi, M. Sinkhorn distances: Lightspeed computation of optimal transport. In *Proc. Neural Information Processing Systems (NeurIPS)*, 2013.
- Cuturi, M. and Blondel, M. Soft-DTW: A Differentiable Loss Function for Time-Series. In *Proc. International Conference on Machine Learning (ICML)*, 2017.
- Cuturi, M., Teboul, O., and Vert, J.-P. Differentiable ranking and sorting using optimal transport. In *Proc. Neural Information Processing Systems (NeurIPS)*, 2019.
- De Smet, L., Sansone, E., and Zuidberg Dos Martires, P. Differentiable sampling of categorical distributions using the catlog-derivative trick. *Advances in Neural Information Processing Systems*, 36, 2024.
- Duchi, J. C., Bartlett, P. L., and Wainwright, M. J. Randomized smoothing for stochastic optimization. *SIAM Journal on Optimization*, 22(2):674–701, 2012.
- Ferguson, T. S. A representation of the symmetric bivariate cauchy distribution. *The Annals of Mathematical Statistics*, 33:1256–1266, 1962.
- Fischer, M. and Ritschel, T. Plateau-reduced differentiable path tracing. In *Proc. International Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4285–4294, 2023a.
- Fischer, M. and Ritschel, T. Zero grads: Learning local surrogate losses for non-differentiable graphics. *Computing Research Repository (CoRR) in arXiv*, 2023b.
- Flaxman, A. D., Kalai, A. T., and McMahan, H. B. Online convex optimization in the bandit setting: gradient descent without a gradient. *arXiv preprint cs/0408007*, 2004.
- Fu, M. C. Gradient estimation. *Handbooks in operations research and management science*, 13:575–616, 2006.
- Glasserman, P. *Gradient estimation via perturbation analysis*, volume 116. Springer Science & Business Media, 1990.
- Grover, A., Wang, E., Zweig, A., and Ermon, S. Stochastic Optimization of Sorting Networks via Continuous Relaxations. In *Proc. International Conference on Learning Representations (ICLR)*, 2019.
- Hammersley, J. M. and Morton, K. W. A new Monte Carlo technique: antithetic variates. *Mathematical proceedings of the Cambridge philosophical society*, 52:449–475, 1956.
- Kagan, M. and Heinrich, L. Branches of a tree: Taking derivatives of programs with discrete and branching randomness in high energy physics. *arXiv preprint arXiv:2308.16680*, 2023.
- Kato, H., Ushiku, Y., and Harada, T. Neural 3D mesh renderer. In *Proc. International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- Kato, H., Beker, D., Morariu, M., Ando, T., Matsuoka, T., Kehl, W., and Gaidon, A. Differentiable rendering: A survey. *Computing Research Repository (CoRR) in arXiv*, 2020.
- Krieken, E., Tomczak, J., and Ten Teije, A. Stochastic: A framework for general stochastic automatic differentiation. *Advances in Neural Information Processing Systems*, 34:7574–7587, 2021.

- LeCun, Y., Cortes, C., and Burges, C. Mnist handwritten digit database. 2010. URL <http://yann.lecun.com/exdb/mnist>.
- Lidec, Q. L., Laptev, I., Schmid, C., and Carpentier, J. Differentiable rendering with perturbed optimizers. In *Proc. Neural Information Processing Systems (NeurIPS)*, 2021.
- Liu, S., Li, T., Chen, W., and Li, H. Soft Rasterizer: A Differentiable Renderer for Image-based 3D Reasoning. In *Proc. International Conference on Computer Vision (ICCV)*, 2019.
- Loper, M. M. and Black, M. J. OpenDR: An approximate differentiable renderer. In *Proc. European Conference on Computer Vision (ECCV)*, 2014.
- L’Ecuyer, P. *Randomized quasi-Monte Carlo: An introduction for practitioners*. Springer, 2018.
- McKay, M. D., Beckman, R. J., and Conover, W. J. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics*, 21:239–245, 1979.
- Metropolis, N. and Ulam, S. M. The monte carlo method. *Journal of the American Statistical Association*, 44:335–341, 1949.
- Mimori, T. and Hamada, M. Geophy: differentiable phylogenetic inference via geometric gradients of tree topologies. *Advances in Neural Information Processing Systems*, 2023.
- Niederreiter, H. Quasi-Monte Carlo methods and pseudo-random numbers. *Bulletin of the American Mathematical Society*, 84:957–1041, 1978.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. Pytorch: An imperative style, high-performance deep learning library. In *Proc. Neural Information Processing Systems (NeurIPS)*. 2019.
- Petersen, F. *Learning with Differentiable Algorithms*. PhD thesis, Universität Konstanz, 2022.
- Petersen, F., Bermano, A. H., Deussen, O., and Cohen-Or, D. Pix2Vex: Image-to-Geometry Reconstruction using a Smooth Differentiable Renderer. *Computing Research Repository (CoRR) in arXiv*, 2019.
- Petersen, F., Borgelt, C., Kuehne, H., and Deussen, O. Differentiable sorting networks for scalable sorting and ranking supervision. In *Proc. International Conference on Machine Learning (ICML)*, 2021a.
- Petersen, F., Borgelt, C., Kuehne, H., and Deussen, O. Learning with algorithmic supervision via continuous relaxations. In *Proc. Neural Information Processing Systems (NeurIPS)*, 2021b.
- Petersen, F., Borgelt, C., Kuehne, H., and Deussen, O. Monotonic differentiable sorting networks. In *Proc. International Conference on Learning Representations (ICLR)*, 2022a.
- Petersen, F., Goldluecke, B., Borgelt, C., and Deussen, O. GenDR: A Generalized Differentiable Renderer. In *Proc. International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022b.
- Petersen, F., Goldluecke, B., Deussen, O., and Kuehne, H. Style agnostic 3d reconstruction via adversarial style transfer. In *IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2022c.
- Petersen, F., Cuturi, M., Niepert, M., Kuehne, H., Kagan, M., Neiswanger, W., and Ermon, S. Differentiable Almost Everything: Differentiable Relaxations, Algorithms, Operators, and Simulators Workshop at ICML 2023. 2023.
- Petersen, F., Mishra, A., Kuehne, H., Borgelt, C., Deussen, O., and Yurochkin, M. Uncertainty quantification via stable distribution propagation. In *Proc. International Conference on Learning Representations (ICLR)*, 2024.
- Prillo, S. and Eisenschlos, J. Softsort: A continuous relaxation for the argsort operator. In *Proc. International Conference on Machine Learning (ICML)*, 2020.
- Quenouille, M. H. Notes on bias in estimation. *Biometrika*, 43:353–360, 1956.
- Rullgård, H., Öfverstedt, L.-G., Masich, S., Daneholt, B., and Öktem, O. Simulation of transmission electron microscope images of biological specimens. *Journal of microscopy*, 243(3):234–256, 2011.
- Sachse, C., Chen, J. Z., Coureux, P.-D., Stroupe, M. E., Fändrich, M., and Grigorieff, N. High-resolution electron microscopy of helical specimens: a fresh look at tobacco mosaic virus. *Journal of molecular biology*, 371(3):812–835, 2007.
- Sander, M. E., Puigcerver, J., Djolonga, J., Peyré, G., and Blondel, M. Fast, differentiable and sparse top-k: a convex analysis perspective. In *Proc. International Conference on Machine Learning (ICML)*, 2023.
- Schmidhuber, J. *Making the world differentiable: on using self supervised fully recurrent neural networks for dynamic reinforcement learning and planning in non-stationary environments*, volume 126. Inst. für Informatik, 1990.

- Showalter, R. E. *Hilbert space methods in partial differential equations*. Courier Corporation, 2010.
- Shvetsova, N., Petersen, F., Kukleva, A., Schiele, B., and Kuehne, H. Learning by sorting: Self-supervised learning with group ordering constraints. In *Proc. International Conference on Computer Vision (ICCV)*, 2023.
- Stewart, L., Bach, F. S., López, F. L., and Berthet, Q. Differentiable clustering with perturbed spanning forests. *Proc. Neural Information Processing Systems (NeurIPS)*, 2023.
- Suh, H. J., Simchowitz, M., Zhang, K., and Tedrake, R. Do differentiable simulators give better policy gradients? In *Proc. International Conference on Machine Learning (ICML)*, 2022.
- Sutton, R. and Barto, A. *Reinforcement Learning, second edition: An Introduction*. Adaptive Computation and Machine Learning series. MIT Press, 2018. ISBN 9780262039246.
- Tukey, J. W. Bias and confidence in not quite large samples. *The Annals of Mathematical Statistics*, 29:614, 1958.
- Vauvelle, A., Wild, B., Eils, R., and Denaxas, S. Differentiable sorting for censored time-to-event data. In *ICML 2023 Workshop on Differentiable Almost Everything: Differentiable Relaxations, Algorithms, Operators, and Simulators*, 2023.
- Vlastelica, M., Paulus, A., Musil, V., Martius, G., and Rolinek, M. Differentiation of blackbox combinatorial solvers. In *Proc. International Conference on Learning Representations (ICLR)*, 2020.
- Wigner, E. P. Characteristic vectors of bordered matrices with infinite dimensions. *Annals of Mathematics*, 62: 548–564, 1955.
- Wigner, E. P. On the distribution of the roots of certain symmetric matrices. *Annals of Mathematics*, 67:325–328, 1958.
- Williams, D. B. and Carter, C. B. *The transmission electron microscope*. Springer, 1996.
- Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8:229–256, 1992.
- Yousefian, F., Nedić, A., and Shanbhag, U. V. Convex non-differentiable stochastic optimization: A local randomized smoothing technique. In *Proceedings of the 2010 American Control Conference*, pp. 4875–4880. IEEE, 2010.

A. Proofs

A.1. Proof of Lemma 2.3

Proof of Lemma 2.3. Let $\Omega \subset \mathbb{R}^n$ be the set of values where $\nabla_\epsilon \mu(\epsilon)$ is undefined. μ is differentiable a.e. and Ω has Lebesgue measure 0.

Recapitulating (5) from the proof of Lemma 2.1, we have

$$\nabla_x f_\epsilon(x) = - \int f(x + \epsilon) \nabla_\epsilon \mu(\epsilon) d\epsilon. \quad (14)$$

Replacing $\nabla_\epsilon \mu(\epsilon)$ by *any* of the weak derivatives ν of μ , which exists and is integrable due to absolute continuity, we have

$$\nabla_x f_\epsilon(x) = - \int f(x + \epsilon) \nu(\epsilon) d\epsilon \quad (15)$$

$$= - \int_{\mathbb{R}^n \setminus \Omega} f(x + \epsilon) \nu(\epsilon) d\epsilon - \int_{\Omega} f(x + \epsilon) \nu(\epsilon) d\epsilon. \quad (16)$$

Because μ is absolutely continuous and as the Lebesgue measure of Ω is 0, per Hölder's inequality

$$\int_{\Omega} |f(x + \epsilon) \nu(\epsilon)| d\epsilon \leq \int_{\Omega} |f(x + \epsilon)| d\epsilon \cdot \int_{\Omega} |\nu(\epsilon)| d\epsilon = \int_{\Omega} |f(x + \epsilon)| d\epsilon \cdot 0 = 0 \quad (17)$$

where $\int_{\Omega} |\nu(\epsilon)| d\epsilon = 0$ follows from absolute continuity of μ . Thus,

$$\int_{\Omega} f(x + \epsilon) \nu(\epsilon) d\epsilon = 0. \quad (18)$$

As $\nu = \nabla_\epsilon \mu(\epsilon)$ for all $\epsilon \in \mathbb{R}^n \setminus \Omega$

$$\nabla_x f_\epsilon(x) = - \int_{\mathbb{R}^n \setminus \Omega} f(x + \epsilon) \nu(\epsilon) d\epsilon - \int_{\Omega} f(x + \epsilon) \nu(\epsilon) d\epsilon = - \int_{\mathbb{R}^n \setminus \Omega} f(x + \epsilon) \nabla_\epsilon \mu(\epsilon) d\epsilon, \quad (19)$$

showing that for all possible choices of ν , the gradient estimator coincides. Thus, we complete our proof via

$$\nabla_x f_\epsilon(x) = - \int_{\mathbb{R}^n \setminus \Omega} f(x + \epsilon) \mu(\epsilon) \nabla_\epsilon \log \mu(\epsilon) d\epsilon = \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \epsilon) \cdot \mathbf{1}_{\epsilon \notin \Omega} \cdot \nabla_\epsilon \log \mu(\epsilon) \right]. \quad (20)$$

After completing the proof, we remark that, if the density was not continuous, e.g., uniform $\mathcal{U}([0, 1])$, then $\int_{\{0\}} \nabla_\epsilon \mu(\epsilon) d\epsilon = \left[\mu(\epsilon) \right]_{\epsilon \nearrow 0}^{\epsilon \searrow 0} = 1$. This means that the weak derivative is not defined (or loosely speaking “the derivative is infinity”), thereby violating the assumptions of Hölder's inequality (Eq. 17). This concludes that continuity is required for the proof to hold. $\square$

A.2. Proof of Lemma 2.6

Proof of Lemma 2.6.

$$\nabla_\gamma f_{\gamma\epsilon}(x) = \nabla_\gamma \mathbb{E}_{\epsilon \sim \mu} [f(x + \gamma \cdot \epsilon)] \quad (21)$$

$$= \nabla_\gamma \int f(x + \gamma \cdot \epsilon) \mu(\epsilon) d\epsilon \quad (22)$$

$$(u = x + \epsilon \cdot \gamma \Rightarrow \epsilon = \frac{u-x}{\gamma}; \frac{du}{d\epsilon} = \gamma \Rightarrow d\epsilon = \frac{1}{\gamma} du) \quad (23)$$

$$= \nabla_\gamma \int f(u) \mu(\epsilon) \frac{1}{\gamma} du \quad (24)$$

$$= \int f(u) \nabla_\gamma (\mu(\epsilon) \frac{1}{\gamma}) du \quad (25)$$

$$= \int f(u) (\frac{1}{\gamma} \nabla_\gamma \mu(\epsilon) + \mu(\epsilon) \nabla_\gamma \frac{1}{\gamma}) du \quad (26)$$

$$= \int f(u) (\frac{1}{\gamma} (\nabla_\epsilon \mu(\epsilon))^\top \frac{\partial \epsilon}{\partial \gamma} - \mu(\epsilon) \frac{1}{\gamma^2}) du \quad (27)$$

$$= \int f(u) (\frac{1}{\gamma} (\nabla_\epsilon \mu(\epsilon))^\top \frac{\partial}{\partial \gamma} \frac{u-x}{\gamma} - \mu(\epsilon) \frac{1}{\gamma^2}) du \quad (28)$$

$$= \int f(u) (\frac{1}{\gamma} (\nabla_\epsilon \mu(\epsilon))^\top (-\frac{\epsilon}{\gamma}) - \mu(\epsilon) \frac{1}{\gamma^2}) du \quad (29)$$

$$= \int f(u) (-(\nabla_\epsilon \mu(\epsilon))^\top \epsilon - \mu(\epsilon) \frac{1}{\gamma^2}) du \quad (30)$$

$$(\nabla_\epsilon \log \mu(\epsilon) = \frac{1}{\mu(\epsilon)} \nabla_\epsilon \mu(\epsilon) \Rightarrow \nabla_\epsilon \mu(\epsilon) = \mu(\epsilon) \nabla_\epsilon \log \mu(\epsilon)) \quad (31)$$

$$= \int f(u) (-(\mu(\epsilon) \nabla_\epsilon \log \mu(\epsilon))^\top \epsilon - \mu(\epsilon) \frac{1}{\gamma^2}) \cdot \underbrace{\gamma d\epsilon}_{=du} \quad (32)$$

$$= \int f(u) \cdot (-(\nabla_\epsilon \log \mu(\epsilon))^\top \epsilon - 1) \cdot \frac{1}{\gamma} \cdot \mu(\epsilon) d\epsilon \quad (33)$$

$$= \int f(u) \cdot (-1 + (\nabla_\epsilon \log \mu(\epsilon))^\top \epsilon) \cdot \frac{1}{\gamma} \cdot \mu(\epsilon) d\epsilon \quad (34)$$

$$= \mathbb{E}_{\epsilon \sim \mu} [f(x + \gamma \cdot \epsilon) \cdot (-1 + (\nabla_\epsilon \log \mu(\epsilon))^\top \epsilon) / \gamma]. \quad (35)$$

□

A.3. Proof of Theorem 2.7

Proof of Theorem 2.7.

Part 1: $\partial f_{\mathbf{L}\epsilon}(x) / \partial x$

We perform a change of variables, $u = x + \mathbf{L}\epsilon \implies \epsilon = \mathbf{L}^{-1}(u - x)$ and

$$d\epsilon = \frac{du}{d\mathbf{L}} d\epsilon = \frac{d\epsilon}{du} du = \frac{d\mathbf{L}^{-1}(u - x)}{du} du = \frac{d\mathbf{L}^{-1}u}{du} du = \det(\mathbf{L}^{-1}) du \quad (36)$$

Thus,

$$f_{\mathbf{L}\epsilon}(x) = \int f(x + \mathbf{L}\epsilon) \mu(\epsilon) d\epsilon = \int f(u) \cdot \mu(\mathbf{L}^{-1}(u - x)) \cdot \det(\mathbf{L}^{-1}) du. \quad (37)$$

Now,

$$\nabla_x f_{\mathbf{L}\epsilon}(x)_i = \nabla_x \int f(u)_i \cdot \mu(\mathbf{L}^{-1}(u - x)) \cdot \det(\mathbf{L}^{-1}) du \quad (38)$$

$$= \int f(u)_i \cdot \nabla_x (\mu(\mathbf{L}^{-1}(u - x))) \cdot \det(\mathbf{L}^{-1}) du \quad (39)$$

$$= \int f(u)_i \cdot \mathbf{L}^{-1} \cdot (\nabla_{\epsilon} \mu(\epsilon)) \cdot \det(\mathbf{L}^{-1}) du \quad (40)$$

$$= \int f(x + \mathbf{L}\epsilon)_i \cdot \mathbf{L}^{-1} \cdot \nabla_{\epsilon} \mu(\epsilon) d\epsilon \quad (41)$$

$$= \int f(x + \mathbf{L}\epsilon)_i \cdot \mathbf{L}^{-1} \cdot \mu(\epsilon) \cdot \nabla_{\epsilon} \log \mu(\epsilon) d\epsilon \quad (42)$$

$$= \mathbb{E}_{\epsilon \sim \mu} [f(x + \mathbf{L}\epsilon)_i \cdot \mathbf{L}^{-1} \cdot \nabla_{\epsilon} \log \mu(\epsilon)] \quad (43)$$

Part 2: $\partial f_{\mathbf{L}\epsilon}(x) / \partial \mathbf{L}$

We use the same change of variables as above.

$$\nabla_{\mathbf{L}} \mathbb{E}_{\epsilon \sim \mu} [f(x + \mathbf{L} \cdot \epsilon)_i] \quad (44)$$

$$= \nabla_{\mathbf{L}} \int f(x + \mathbf{L}\epsilon)_i \cdot \mu(\epsilon) d\epsilon \quad (45)$$

$$= \nabla_{\mathbf{L}} \int f(u)_i \cdot \mu(\mathbf{L}^{-1}(u - x)) \cdot \det(\mathbf{L}^{-1}) du \quad (46)$$

$$= \int f(u)_i \cdot \nabla_{\mathbf{L}} (\mu(\mathbf{L}^{-1}(u - x)) \cdot \det(\mathbf{L}^{-1})) du \quad (47)$$

$$= \int f(x + \mathbf{L}\epsilon)_i \cdot \nabla_{\mathbf{L}} (\mu(\mathbf{L}^{-1}(u - x)) \cdot \det(\mathbf{L}^{-1})) / \det(\mathbf{L}^{-1}) d\epsilon \quad (48)$$

$$= \mathbb{E}_{\epsilon \sim \mu} [f(x + \mathbf{L}\epsilon)_i \cdot \nabla_{\mathbf{L}} (\mu(\mathbf{L}^{-1}(u - x)) \cdot \det(\mathbf{L}^{-1})) \cdot \det(\mathbf{L}) / \mu(\epsilon)] \quad (49)$$

Now, while $\nabla_{\mathbf{L}} (\mu(\mathbf{L}^{-1}(u - x)) \cdot \det(\mathbf{L}^{-1}))$ may be computed via automatic differentiation, we can also solve it in closed-form. Firstly, we can observe that

$$\nabla_{\mathbf{L}} \mu(\mathbf{L}^{-1}(u - x)) = \nabla_{\epsilon}^{\top} \mu(\epsilon) \cdot \nabla_{\mathbf{L}} (\mathbf{L}^{-1} \cdot (u - x)) \quad (50)$$

$$= \nabla_{\mathbf{L}} (\nabla_{\epsilon}^{\top} \mu(\epsilon) \cdot \mathbf{L}^{-1} \cdot (u - x)) \quad (51)$$

$$= -\mathbf{L}^{-\top} \cdot \nabla_{\epsilon} \mu(\epsilon) \cdot (u - x)^{\top} \cdot \mathbf{L}^{-\top} \quad (52)$$

and

$$\nabla_{\mathbf{L}} \det(\mathbf{L}^{-1}) = -\det(\mathbf{L})^{-1} \cdot \mathbf{L}^{-\top}. \quad (53)$$

We can combine this to resolve it in closed form to:

$$\begin{aligned} \nabla_{\mathbf{L}} (\mu(\mathbf{L}^{-1}(u - x)) \cdot \det(\mathbf{L}^{-1})) &= -\mathbf{L}^{-\top} \cdot \nabla_{\epsilon} \mu(\epsilon) \cdot (u - x)^{\top} \cdot \mathbf{L}^{-\top} \cdot \det(\mathbf{L}^{-1}) \\ &\quad - \mu(\mathbf{L}^{-1}(u - x)) \cdot \det(\mathbf{L})^{-1} \cdot \mathbf{L}^{-\top} \end{aligned} \quad (54)$$

$$\begin{aligned} &= -\mathbf{L}^{-\top} \cdot \nabla_{\epsilon} \mu(\epsilon) \cdot (\mathbf{L}^{-1}(u - x))^{\top} \cdot \det(\mathbf{L}^{-1}) \\ &\quad - \mu(\epsilon) \cdot \det(\mathbf{L})^{-1} \cdot \mathbf{L}^{-\top} \end{aligned} \quad (55)$$

$$\begin{aligned} &= -\mathbf{L}^{-\top} \cdot \nabla_{\epsilon} \mu(\epsilon) \cdot \epsilon^{\top} \cdot \det(\mathbf{L}^{-1}) \\ &\quad - \mu(\epsilon) \cdot \det(\mathbf{L})^{-1} \cdot \mathbf{L}^{-\top} \end{aligned} \quad (56)$$

$$= -\det(\mathbf{L}^{-1}) \cdot (\mathbf{L}^{-\top} \cdot \nabla_{\epsilon} \mu(\epsilon) \cdot \epsilon^{\top} + \mu(\epsilon) \cdot \mathbf{L}^{-\top}). \quad (57)$$

Combing this with equation (49), we have

$$\begin{aligned} & \nabla_{\mathbf{L}} \mathbb{E}_{\epsilon \sim \mu} [f(x + \mathbf{L} \cdot \epsilon)_i] \\ &= \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot \nabla_{\mathbf{L}} \left(\mu(\mathbf{L}^{-1}(u - x)) \cdot \det(\mathbf{L}^{-1}) \right) \cdot \det(\mathbf{L}) / \mu(\epsilon) \right] \\ &= \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot -\det(\mathbf{L}^{-1}) \cdot (\mathbf{L}^{-\top} \cdot \nabla_{\epsilon} \mu(\epsilon) \cdot \epsilon^{\top} + \mu(\epsilon) \cdot \mathbf{L}^{-\top}) \cdot \det(\mathbf{L}) / \mu(\epsilon) \right] \end{aligned} \quad (58)$$

$$= \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot -(\mathbf{L}^{-\top} \cdot \nabla_{\epsilon} \mu(\epsilon) \cdot \epsilon^{\top} + \mu(\epsilon) \cdot \mathbf{L}^{-\top}) / \mu(\epsilon) \right] \quad (59)$$

$$= \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot -(\mathbf{L}^{-\top} \cdot \nabla_{\epsilon} \mu(\epsilon) \cdot \epsilon^{\top} / \mu(\epsilon) + \mathbf{L}^{-\top}) \right] \quad (60)$$

$$= \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot \mathbf{L}^{-\top} \cdot (-1 + \nabla_{\epsilon} - \log \mu(\epsilon) \cdot \epsilon^{\top}) \right]. \quad (61)$$

□

B. Discussion of Properties of f for Finitely Defined f_{ϵ} and ∇f_{ϵ}

When we have a function f that is not defined with a compact range with $f : \mathbb{R}^n \rightarrow \mathbb{R}$, and have a density μ with unbounded support (e.g., Gaussian or Cauchy), we may experience f_{ϵ} or even ∇f_{ϵ} to not be finitely defined. For example, virtually any distribution with full support on $\mathbb{R}$ leads to the smoothing f_{ϵ} of the degenerate function $f : x \mapsto \exp(\exp(\exp(\exp(x^2))))$ to not be finitely defined.

We say a function, as described via an expectation, is finitely defined iff it is defined (i.e., the expectation has a value) and its value is finite (i.e., not infinity). For example, the first moment of the Cauchy distribution is undefined, and the second moment is infinite; thus, both moments are not finitely defined.

We remark that the considerations in this appendix also apply to prior works that enable the real plane as the output space of f . We further remark that writing an expression for smoothing and the gradient of a arbitrary function with non-compact range is not necessarily false; however, e.g., any claim that smoothness is guaranteed if the gradient jumps from $-\infty$ to ∞ (e.g., the power tower in the first paragraph) is not formally correct. We remark that characterizing valid f s via a Lipschitz or other continuity requirement is not applicable because this would defeat the goal of differentiating non-differentiable and discontinuous f .

In the following, we discuss when f_{ϵ} or ∇f_{ϵ} are finitely defined. For this, let us cover a few preliminaries:

Let a function $f(x)$ be called $\mathcal{O}(b(x))$ bounded if there exist $c, v \in \mathcal{O}(b(x))$ and $\bar{c}, \bar{v} \in \mathbb{R}$ such that

$$\bar{c} + c(x) \leq f(x) \leq \bar{v} + v(x) \quad \forall x. \quad (62)$$

For example, a function may be called polynomially bounded (wrt. a polynomial $b(x)$) if (but not only if) $-b(x) \leq f(x) \leq b(x)$.

Moreover, let a density μ with support $\mathbb{R}$ be called decaying faster than $b(x)$ if $\mu(x) \in o(b(x))$. For example, the standard Gaussian density decays faster than $\exp(-|x|)$, i.e., $\mu(x) \in o(\exp(-|x|))$. Additionally, we can say that Gaussian density decays at rate $\exp(-x^2)$, i.e., $\mu(x) \in \theta(\exp(-x^2))$.

Now, we can formally characterize finite definedness of f_{ϵ} and ∇f_{ϵ} :

Lemma B.1 (Finite Definedness of f_{ϵ}). *f_{ϵ} is finitely defined if there exists an increasing function $b(\cdot)$ such that*

$$f(x) \text{ is bounded by } \mathcal{O}(b(x)) \quad \text{and} \quad \mu(\epsilon) \in \mathcal{O}(1/b(\epsilon + \alpha\epsilon)/\epsilon^{(1+\alpha)}) \quad (63)$$

for some $\alpha > 0$.

Proof. To show that f_{ϵ} exists, we need to show that

$$\int_{\mathbb{R}} |f(x + \epsilon) \cdot \mu(\epsilon)| d\epsilon \quad (64)$$

is finite for all x . Let $\tilde{f}$ be an absolutely upper bound of f , and w.l.o.g. let us choose $\tilde{f}(y) = b(y) + \bar{b}$ with $b(y) > 1$ for $y \in \mathbb{R}$. Further, as per the assumptions $\mu(\epsilon) < \frac{1}{\epsilon^{(1+\alpha)} \cdot b(\epsilon + \alpha\epsilon)} \cdot w$ for all $\epsilon < \omega_1$ as well as all $\epsilon > \omega_2$ for some w, ω_1, ω_2 . Let us restrict ω_1, ω_2 to $\omega_1 < -|x|/\alpha$ and $\omega_2 > |x|/\alpha$. It is trivial to see that

$$\int_{\omega_1}^{\omega_2} |f(x + \epsilon) \cdot \mu(\epsilon)| d\epsilon < \infty. \quad (65)$$

W.l.o.g., let us consider the upper remainder:

$$\int_{\omega_2}^{\infty} |f(x + \epsilon) \cdot \mu(\epsilon)| d\epsilon \leq \int_{\omega_2}^{\infty} |\tilde{f}(x + \epsilon) \cdot \mu(\epsilon)| d\epsilon \quad (66)$$

$$\leq \int_{\omega_2}^{\infty} \left| (b(x + \epsilon) + \bar{b}) \cdot \frac{1}{\epsilon^{(1+\alpha)} \cdot b(\epsilon + \alpha\epsilon)} \cdot w \right| d\epsilon \quad (67)$$

$$= \int_{\omega_2}^{\infty} \left| \left(\frac{b(x + \epsilon)}{\epsilon^{(1+\alpha)} \cdot b(\epsilon + \alpha\epsilon)} + \frac{\bar{b}}{\epsilon^{(1+\alpha)} \cdot b(\epsilon + \alpha\epsilon)} \right) \cdot w \right| d\epsilon \quad (68)$$

$$\leq \int_{\omega_2}^{\infty} \left| \left(\frac{b(x + \epsilon)}{\epsilon^{(1+\alpha)} \cdot b(\epsilon + |x|)} + \frac{\bar{b}}{\epsilon^{(1+\alpha)} \cdot b(\epsilon + \alpha\epsilon)} \right) \cdot w \right| d\epsilon \quad (69)$$

$$\leq \int_{\omega_2}^{\infty} \left| \left(\frac{1}{\epsilon^{(1+\alpha)}} + \frac{\bar{b}}{\epsilon^{(1+\alpha)} \cdot b(\epsilon + \alpha\epsilon)} \right) \cdot w \right| d\epsilon \quad (70)$$

$$< \int_{\omega_2}^{\infty} \left| \frac{1}{\epsilon^{(1+\alpha)}} + \frac{\bar{b}}{\epsilon^{(1+\alpha)}} \right| d\epsilon \cdot w \quad (71)$$

$$= \int_{\omega_2}^{\infty} \left| \frac{1}{\epsilon^{(1+\alpha)}} \right| d\epsilon \cdot w \cdot (1 + \bar{b}) < \infty. \quad (72)$$

That $\int_{\omega_2}^{\infty} \frac{1}{\epsilon^{(1+\alpha)}} d\epsilon$ is finite for the step in (72) can be shown via

$$\int_{\omega_2}^{\infty} \frac{1}{\epsilon^{(1+\alpha)}} d\epsilon = \int_{\omega_2}^{\infty} \epsilon^{-1-\alpha} d\epsilon = \left[-\frac{1}{\alpha} \epsilon^{-\alpha} \right]_{\omega_2}^{\infty} = \left[-\frac{1}{\alpha} \lim_{\epsilon \rightarrow \infty} \epsilon^{-\alpha} + \frac{1}{\alpha} \omega_2^{-\alpha} \right] = \frac{1}{\alpha} \omega_2^{-\alpha}.$$

The same can be shown analogously for the integral $\int_{-\infty}^{\omega_1}$. This completes the proof. $\square$

Lemma B.2 (Finite Definedness of ∇f_ϵ). ∇f_ϵ is finitely defined if there exists an increasing function $b(\cdot)$ such that

$$f(x) \text{ is bounded by } \mathcal{O}(b(x)) \quad \text{and} \quad |\mu(\epsilon) \cdot \nabla_\epsilon - \log \mu(\epsilon)| \in \mathcal{O}(1/b(\epsilon + \alpha\epsilon)/\epsilon^{(1+\alpha)}) \quad (73)$$

for some $\alpha > 0$.

Proof. The proof of Lemma B.1 also applies here, but with $|\mu(\epsilon) \cdot \nabla_\epsilon - \log \mu(\epsilon)| < \frac{1}{\epsilon^{(1+\alpha)} \cdot b(\epsilon + \alpha\epsilon)} \cdot w$ for all $\epsilon < \omega_1$ as well as all $\epsilon > \omega_2$ for some w, ω_1, ω_2 . $\square$

Example B.3 (Cauchy and the Identity). Let μ be the density of a Cauchy distribution and let $f(x) = x$. The tightest b for $f(x) \in \mathcal{O}(b(x))$ is $b(x) = x$.

We have $\mu(\epsilon) \in \theta(1/\epsilon^2)$ and thus $\mu(\epsilon) \notin o(1/\epsilon^2)$. f_ϵ , i.e., the mean of the Cauchy distribution is not defined.

However, its gradient $\nabla f_\epsilon = 1$ is indeed finitely defined. In particular, we can see that

$$\mu(\epsilon) \cdot \nabla_\epsilon - \log \mu(\epsilon) = \frac{2\epsilon}{\pi \cdot (1 + \epsilon^2) \cdot (1 + \epsilon^2)} \in \mathcal{O}(1/\epsilon^3). \quad (74)$$

This is an intriguing property of the Cauchy distribution (or other edge cases) where f_ϵ is undefined whereas ∇f_ϵ is finitely and well-defined. In practice, we often only require the gradient for stochastic gradient descent, which means that we often only require ∇f_ϵ to be well defined and do not necessarily need to evaluate f_ϵ depending on the application.

Additional discussions for the Cauchy distribution and an extension of stochastic smoothing to the k -sample median can be found in the next appendix.

C. Stochastic Smoothing, Medians, and the Cauchy Distribution

In this section, we provide a discussion of a special case of stochastic smoothing with the Cauchy distribution, and provide an extension of stochastic smoothing to the k -sample median. This becomes important if the range of f is not subset of a compact set, and thus $\mathbb{E}_{\epsilon \sim \mu}[f(x + \epsilon)]$ becomes undefined for some choice of distribution μ . For example, for $f(x + \epsilon) = \epsilon$ and μ being the density of a Cauchy distribution, $\mathbb{E}_{\epsilon \sim \mu}[f(x + \epsilon)] = \mathbb{E}_{\epsilon \sim \mu}[\epsilon]$ is undefined. Nevertheless, even in this case, the gradient estimators discussed in this paper for $\nabla_x \mathbb{E}_{\epsilon \sim \mu}[f(x + \epsilon)]$ remain well defined. This is practically relevant because $\mathbb{E}_{\epsilon \sim \mu}[f(x + \epsilon)]$ does not need to be finitely defined as long as $\nabla_x \mathbb{E}_{\epsilon \sim \mu}[f(x + \epsilon)]$ is well defined. Further, we remark that the undefinedness of $\mathbb{E}_{\epsilon \sim \mu}[f(x + \epsilon)]$ requires the range of f to be unbounded, i.e., if there exists a maximum / minimum possible output, then it is well defined. Moreover, there exist f with unbounded range for which $\mathbb{E}_{\epsilon \sim \mu}[f(x + \epsilon)]$ also remains well defined.

To account for cases where $\mathbb{E}_{\epsilon \sim \mu}[f(x + \epsilon)]$ may not be well defined or not a robust statistic, we introduce an extension of smoothing to the median. We begin by defining the k -sample median.

Definition C.1 (k -Sample Median). For a number of samples $k > 1$, and a distribution ζ , we say that

$$\mathbb{E}_{z_1, z_2, \dots, z_k \sim \zeta} \left[\text{median} \{z_1, z_2, \dots, z_k\} \right] \quad (75)$$

is the k -sample median. For multivariate distributions, let median be the per-dimension median.

Indeed, for $k \geq 5$, the k -sample median estimator is shown to have finite variance for the Cauchy distribution (Theorem 3 and Example 2 in (Chu & Hotelling, 1955)), which implies a well defined k -sample median. Moreover, for any distribution with a density of the median bounded away from 0, the first and second moments are guaranteed to be finitely defined for sufficiently large k . This is important for non-trivial f with $f(\epsilon) \neq \epsilon$ for at least one ϵ with $\epsilon \sim \mu$, which implies $\zeta \neq \mu$. Thus, rather than computing and differentiating the expected value, we can differentiate the k -sample median.

Lemma C.2 (Differentiation of the k -Sample Median). *With the k -sample median smoothing as*

$$f_\epsilon^{(k)}(x) = \mathbb{E}_{\epsilon_1, \dots, \epsilon_k \sim \mu} \left[\text{median} \{f(x + \epsilon_1), \dots, f(x + \epsilon_k)\} \right], \quad (76)$$

we can differentiate $f_\epsilon^{(k)}(x)$ as

$$\nabla_x f_\epsilon^{(k)}(x) = \mathbb{E}_{\epsilon_1, \dots, \epsilon_k \sim \mu} \left[f(x + \epsilon_{r(\epsilon)}) \cdot \nabla_{\epsilon_{r(\epsilon)}} - \log \mu(\epsilon_{r(\epsilon)}) \right] \quad (77)$$

where $r(\epsilon)$ is the arg-median of the set $\{f(x + \epsilon_1), \dots, f(x + \epsilon_k)\}$, which is equivalent to the implicit definition via $f(x + \epsilon_{r(\epsilon)}) = \text{median} \{f(x + \epsilon_1), \dots, f(x + \epsilon_k)\}$.

Proof. We denote $\epsilon_{1:k} \sim \mu^{(1:k)}$ such that $\epsilon_{1:k} = [\epsilon_1^\top, \dots, \epsilon_k^\top]^\top$ and $\epsilon_i \sim \mu \forall i \in \{1, \dots, k\}$.

$$\nabla_x f_\epsilon^{(k)}(x) = \nabla_x \mathbb{E}_{\epsilon_1, \dots, \epsilon_k \sim \mu} \left[\text{median} \{f(x + \epsilon_1), \dots, f(x + \epsilon_k)\} \right] \quad (78)$$

$$= \nabla_x \mathbb{E}_{\epsilon_{1:k} \sim \mu^{(1:k)}} \left[\text{median} \{f(x + \epsilon_1), \dots, f(x + \epsilon_k)\} \right] \quad (79)$$

$$= \nabla_x \int_{\mathbb{R}^{n \cdot k}} \text{median} \{f(x + \epsilon_1), \dots, f(x + \epsilon_k)\} \cdot \mu^{(1:k)}(\epsilon_{1:k}) d\epsilon_{1:k} \quad (80)$$

$$(x_1, \dots, x_k = x) = \sum_{j=1}^k \nabla_{x_j} \int_{\mathbb{R}^{n \cdot k}} \text{median} \{f(x_1 + \epsilon_1), \dots, f(x_k + \epsilon_k)\} \cdot \mu^{(1:k)}(\epsilon_{1:k}) d\epsilon_{1:k} \quad (81)$$

As a shorthand, we abbreviate the indicator $\mathbb{1}_{f(x_j + \epsilon_j) = \text{median}\{f(x_1 + \epsilon_1), \dots, f(x_k + \epsilon_k)\}}$ as $\mathbb{1}_{j, \epsilon_{1:k}}$ and abbreviate

$\mathbb{K}_{f(u_j)=\text{median}\{f(u_1),\dots,f(u_k)\}} \text{ as } \mathbb{K}_{j,u_{1:k}}:$

$$\nabla_x f_\epsilon^{(k)}(x) = \sum_{j=1}^k \nabla_{x_j} \int_{\mathbb{R}^{n \cdot k}} f(x_j + \epsilon_j) \cdot \mathbb{K}_{j,\epsilon_{1:k}} \cdot \mu^{(1:k)}(\epsilon_{1:k}) d\epsilon_{1:k} \quad (82)$$

$$= \sum_{j=1}^k \nabla_{x_j} \int_{\mathbb{R}^{n \cdot k}} f(u) \cdot \mathbb{K}_{j,u_{1:k}} \cdot \mu^{(1:k)}(u_{1:k} - x) du_{1:k} \quad (83)$$

$$= \sum_{j=1}^k \int_{\mathbb{R}^{n \cdot k}} f(u) \cdot \mathbb{K}_{j,u_{1:k}} \cdot \nabla_{x_j} \mu^{(1:k)}(u_{1:k} - x) du_{1:k} \quad (84)$$

$$= \sum_{j=1}^k \int_{\mathbb{R}^{n \cdot k}} f(x + \epsilon_j) \cdot \mathbb{K}_{j,\epsilon_{1:k}} \cdot -\nabla_{\epsilon_j} \mu^{(1:k)}(\epsilon_{1:k}) d\epsilon_{1:k} \quad (85)$$

We have

$$\nabla_{\epsilon_j} \mu^{(1:k)}(\epsilon_{1:k}) = \mu^{(1:k)}(\epsilon_{1:k}) \cdot \nabla_{\epsilon_j} \log \mu^{(1:k)}(\epsilon_{1:k}) = \mu^{(1:k)}(\epsilon_{1:k}) \cdot \nabla_{\epsilon_j} \log \mu(\epsilon_j). \quad (86)$$

Thus,

$$\nabla_x f_\epsilon^{(k)}(x) = \sum_{j=1}^k \int_{\mathbb{R}^{n \cdot k}} f(x + \epsilon_j) \cdot \mathbb{K}_{j,\epsilon_{1:k}} \cdot -\mu^{(1:k)}(\epsilon_{1:k}) \cdot \nabla_{\epsilon_j} \log \mu(\epsilon_j) d\epsilon_{1:k} \quad (87)$$

$$= \int_{\mathbb{R}^{n \cdot k}} \sum_{j=1}^k [\mathbb{K}_{j,\epsilon_{1:k}} \cdot f(x + \epsilon_j) \cdot \nabla_{\epsilon_j} - \log \mu(\epsilon_j)] \cdot \mu^{(1:k)}(\epsilon_{1:k}) d\epsilon_{1:k} \quad (88)$$

Indicating the choice of median in dependence of $\epsilon_{1:k}$, we define $r(\epsilon_{1:k})$ s.t. $\mathbb{K}_{r(\epsilon_{1:k}),\epsilon_{1:k}} = 1$. Thus,

$$\nabla_x f_\epsilon^{(k)}(x) = \int_{\mathbb{R}^{n \cdot k}} f(x + \epsilon_{r(\epsilon_{1:k})}) \cdot \nabla_{\epsilon_{r(\epsilon_{1:k})}} - \log \mu(\epsilon_{r(\epsilon_{1:k})}) \cdot \mu^{(1:k)}(\epsilon_{1:k}) d\epsilon_{1:k} \quad (89)$$

$$= \mathbb{E}_{\epsilon_{1:k} \sim \mu^{(1:k)}} [f(x + \epsilon_{r(\epsilon_{1:k})}) \cdot \nabla_{\epsilon_{r(\epsilon_{1:k})}} - \log \mu(\epsilon_{r(\epsilon_{1:k})})] \quad (90)$$

This concludes the proof. $\square$

Empirically, we can estimate $\nabla_x f_\epsilon^{(k)}(x)$ for s propagated samples ($s > k$) without bias as

$$\nabla_x f_\epsilon^{(k)}(x) \triangleq \sum_{i=1}^s [q_i \cdot f(x + \epsilon_i) \cdot \nabla_{\epsilon_i} - \log \mu(\epsilon_i)] \quad \epsilon_1, \dots, \epsilon_s \sim \mu \quad (91)$$

where q_i is the probability of $f(x + \epsilon_i)$ being the median in a subset of k samples, i.e., under uniqueness of g_i s, we have

$$q_i = \frac{\sum_{\{h_1, \dots, h_k\} \subset \{g_1, \dots, g_s\}} \mathbb{K}(g_i = \text{median}\{h_1, \dots, h_k\})}{\binom{s}{k}} \quad g_i := f(x + \epsilon_i). \quad (92)$$

We remark that, in case of non-uniqueness, it is adequate to split the probability among the candidates; however, under non-discreteness assumptions on f (density of $\zeta < \infty$, the converse typically implies the range of f being a subset of a compact set), this almost surely (with probability 1) does not occur.

We have shown that the k -sample median $f_\epsilon^{(k)}(x)$ is differentiable and demonstrated an unbiased gradient estimator for it. A straightforward extension for the case of f being differentiable is differentiating through the median via a $k \rightarrow \infty$ -sample median, e.g., via setting $s = k^2$. The $k \rightarrow \infty$ extension for differentiating through the median itself requires f being differentiable because, for discontinuous f , $f_\epsilon^{(k)}(x)$ is differentiable only for $k < \infty$. (As an illustration, the median of the Heaviside function under a symmetric perturbation μ with density at 0 bounded away from 0 is the exactly the Heaviside function.)

D. Differentiating the Output Covariances

In this section, we describe how we can differentiate output covariances of the smoothing process. This could be particularly helpful for uncertainty quantification (UQ) applications, e.g., in the context of propagating distributions (Petersen et al., 2024).

Theorem D.1 (Output Covariance of Multivariate Smoothing for UQ). *Given the assumptions of Theorem 2.7, we may also be interested in computing the output covariance $G_{\mathbf{L}\epsilon}$:*

$$G_{\mathbf{L}\epsilon}(x) = \text{Cov}_{\epsilon \sim \mu} [f(x + \mathbf{L} \cdot \epsilon)]. \quad (93)$$

We can compute the derivative of the output covariance wrt. the input $\partial G_{\mathbf{L}\epsilon}(x) / \partial x (\in \mathbb{R}^{m \times m \times n})$ as

$$\begin{aligned} \nabla_x G_{\mathbf{L}\epsilon}(x)_{i,j} &= \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot f(x + \mathbf{L}\epsilon)_j \cdot \mathbf{L}^{-1} \cdot \nabla_{\epsilon} (-\log \mu(\epsilon)) \right] \\ &\quad - f_{\mathbf{L}\epsilon}(x)_i \cdot \nabla_x f_{\mathbf{L}\epsilon}(x)_j - f_{\mathbf{L}\epsilon}(x)_j \cdot \nabla_x f_{\mathbf{L}\epsilon}(x)_i \end{aligned} \quad (94)$$

Further, we can compute the derivative wrt. the input scale matrix $\partial G_{\mathbf{L}\epsilon}(x) / \partial \mathbf{L} (\in \mathbb{R}^{m \times m \times n \times n})$ as

$$\begin{aligned} \nabla_{\mathbf{L}} G_{\mathbf{L}\epsilon}(x)_{i,j} &= -\mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot f(x + \mathbf{L}\epsilon)_j \cdot (\mathbf{L}^{-\top} \cdot \nabla_{\epsilon} \mu(\epsilon) \cdot \epsilon^{\top} / \mu(\epsilon) + \mathbf{L}^{-\top}) \right] \\ &\quad - f_{\mathbf{L}\epsilon}(x)_i \cdot \nabla_{\mathbf{L}} f_{\mathbf{L}\epsilon}(x)_j - f_{\mathbf{L}\epsilon}(x)_j \cdot \nabla_{\mathbf{L}} f_{\mathbf{L}\epsilon}(x)_i \end{aligned} \quad (95)$$

Proof. Part 1: $\partial G_{\mathbf{L}\epsilon}(x) / \partial x$ We start by stating some helpful preliminaries:

$$G_{\mathbf{L}\epsilon}(x) = \text{Cov}_{\epsilon \sim \mu}(f(x + \mathbf{L}\epsilon)) = \mathbb{E}_{\epsilon \sim \mu} \left[(f(x + \mathbf{L}\epsilon) - f_{\mathbf{L}\epsilon}(x)) (f(x + \mathbf{L}\epsilon) - f_{\mathbf{L}\epsilon}(x))^{\top} \right] \quad (96)$$

$$= \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon) f(x + \mathbf{L}\epsilon)^{\top} \right] - f_{\mathbf{L}\epsilon}(x) f_{\mathbf{L}\epsilon}(x)^{\top}, \quad (97)$$

$$G_{\mathbf{L}\epsilon}(x)_{i,j} = \text{Cov}_{\epsilon \sim \mu}(f(x + \mathbf{L}\epsilon)_i, f(x + \mathbf{L}\epsilon)_j) \quad (98)$$

$$= \mathbb{E}_{\epsilon \sim \mu} \left[(f(x + \mathbf{L}\epsilon) - f_{\mathbf{L}\epsilon}(x))_i \cdot (f(x + \mathbf{L}\epsilon) - f_{\mathbf{L}\epsilon}(x))_j \right] \quad (99)$$

$$= \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot f(x + \mathbf{L}\epsilon)_j \right] - f_{\mathbf{L}\epsilon}(x)_i \cdot f_{\mathbf{L}\epsilon}(x)_j. \quad (100)$$

We proceed by computing the derivative of the left part of Equation 100.

$$\nabla_x \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot f(x + \mathbf{L}\epsilon)_j \right] \quad (101)$$

$$= \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot f(x + \mathbf{L}\epsilon)_j \cdot \mathbf{L}^{-1} \cdot \nabla_{\epsilon} (-\log \mu(\epsilon)) \right], \quad (102)$$

which is analogous to Equations 38 until 43. Now,

$$\nabla_x G_{\mathbf{L}\epsilon}(x)_{i,j} = \nabla_x \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot f(x + \mathbf{L}\epsilon)_j \right] - \nabla_x (f_{\mathbf{L}\epsilon}(x)_i \cdot f_{\mathbf{L}\epsilon}(x)_j) \quad (103)$$

$$= \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot f(x + \mathbf{L}\epsilon)_j \cdot \mathbf{L}^{-1} \cdot \nabla_{\epsilon} (-\log \mu(\epsilon)) \right] \quad (104)$$

$$- f_{\mathbf{L}\epsilon}(x)_i \cdot \nabla_x f_{\mathbf{L}\epsilon}(x)_j - f_{\mathbf{L}\epsilon}(x)_j \cdot \nabla_x f_{\mathbf{L}\epsilon}(x)_i \quad (105)$$

where $\nabla_x f_{\mathbf{L}\epsilon}(x)$ is defined as in part 1 of the proof of Theorem 2.7.

Part 2: $\partial G_{\mathbf{L}\epsilon}(x) / \partial \mathbf{L}$

$$\nabla_{\mathbf{L}} G_{\mathbf{L}\epsilon}(x)_{i,j} \quad (106)$$

$$= \nabla_{\mathbf{L}} \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot f(x + \mathbf{L}\epsilon)_j \right] - \nabla_{\mathbf{L}} (f_{\mathbf{L}\epsilon}(x)_i \cdot f_{\mathbf{L}\epsilon}(x)_j) \quad (107)$$

$$= \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot f(x + \mathbf{L}\epsilon)_j \cdot \nabla_{\mathbf{L}} \left(\mu(\mathbf{L}^{-1}(u - x)) \cdot \det(\mathbf{L}^{-1}) \right) \cdot \det(\mathbf{L}) / \mu(\epsilon) \right] \quad (108)$$

$$- f_{\mathbf{L}\epsilon}(x)_i \cdot \nabla_{\mathbf{L}} f_{\mathbf{L}\epsilon}(x)_j - f_{\mathbf{L}\epsilon}(x)_j \cdot \nabla_{\mathbf{L}} f_{\mathbf{L}\epsilon}(x)_i \quad (109)$$

where $\nabla_{\mathbf{L}} f_{\mathbf{L}\epsilon}(x)$ is defined as in Theorem 2.7.

Using the closed-form solution for $\nabla_{\mathbf{L}} \left(\mu(\mathbf{L}^{-1}(u - x)) \cdot \det(\mathbf{L}^{-1}) \right)$ from the proof of Theorem 2.7, we can simplify it to

$$\nabla_{\mathbf{L}} G_{\mathbf{L}\epsilon}(x)_{i,j} \tag{110}$$

$$= \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot f(x + \mathbf{L}\epsilon)_j \cdot \nabla_{\mathbf{L}} \left(\mu(\mathbf{L}^{-1}(u - x)) \cdot \det(\mathbf{L}^{-1}) \right) \cdot \det(\mathbf{L}) / \mu(\epsilon) \right] \\ - f_{\mathbf{L}\epsilon}(x)_i \cdot \nabla_{\mathbf{L}} f_{\mathbf{L}\epsilon}(x)_j - f_{\mathbf{L}\epsilon}(x)_j \cdot \nabla_{\mathbf{L}} f_{\mathbf{L}\epsilon}(x)_i \tag{111}$$

$$= - \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot f(x + \mathbf{L}\epsilon)_j \cdot (\mathbf{L}^{-\top} \cdot \nabla_{\epsilon} \mu(\epsilon) \cdot \epsilon^{\top} + \mu(\epsilon) \cdot \mathbf{L}^{-\top}) / \mu(\epsilon) \right] \\ - f_{\mathbf{L}\epsilon}(x)_i \cdot \nabla_{\mathbf{L}} f_{\mathbf{L}\epsilon}(x)_j - f_{\mathbf{L}\epsilon}(x)_j \cdot \nabla_{\mathbf{L}} f_{\mathbf{L}\epsilon}(x)_i \tag{112}$$

$$= - \mathbb{E}_{\epsilon \sim \mu} \left[f(x + \mathbf{L}\epsilon)_i \cdot f(x + \mathbf{L}\epsilon)_j \cdot (\mathbf{L}^{-\top} \cdot \nabla_{\epsilon} \mu(\epsilon) \cdot \epsilon^{\top} / \mu(\epsilon) + \mathbf{L}^{-\top}) \right] \\ - f_{\mathbf{L}\epsilon}(x)_i \cdot \nabla_{\mathbf{L}} f_{\mathbf{L}\epsilon}(x)_j - f_{\mathbf{L}\epsilon}(x)_j \cdot \nabla_{\mathbf{L}} f_{\mathbf{L}\epsilon}(x)_i \tag{113}$$

□

E. Variances and Lipschitz Constants for each Distribution

In this section, we provide variances of the gradient estimator ∇f_ϵ and Lipschitz constants of f_ϵ for each of the 6 distributions considered in the paper.

Table 3. Lipschitz constants for each distribution for functions $f : \mathbb{R}^n \rightarrow \{0, 1\}^m$.

Distribution	Lipschitz constant
Gaussian	$\frac{1}{\sqrt{2\pi}\gamma} \approx 0.399/\gamma$
Logistic	$0.25/\gamma$
Gumbel	$\frac{1}{e\gamma} \approx 0.368/\gamma$
Cauchy	$\frac{1}{\pi\gamma} \approx 0.318/\gamma$
Laplace	$0.5/\gamma$
Triangular	$1.0/\gamma$

In Table 4, we can see that the Laplace distribution performs best, consistently achieving a variance of 0, which however, in this particular case is due to the simple nature of the problem. The second best distribution is the logistic distribution. The logistic behavior is similar to the Laplace distribution also from an analytical perspective (the logistic can be seen as a smoothed variant of the Laplace).

Table 4. Variances in the case of the sign function ($f(x) = \text{sign}(x)$), smoothed with each respective distribution, and evaluating the variance at point 0, in dependence of the number of samples s . To standardize the distributions, we consider 2 settings: scaling each input distribution to a variance of 1, as well as choosing a scale such that the Lipschitz constant of the resulting smooth function is 1.

$f(x) = \text{sign}(x)$	Var = 1	Lipschitz = 1
Gaussian	$0.364/s$	$0.573/s$
Logistic	$0.272/s$	$0.336/s$
Gumbel	$0.759/s$	$0.859/s$
Cauchy	n/a	$0.234/s$
Laplace	$0.000/s$	$0.000/s$
Triangular	$1.747/s$	$2.496/s$

In the more complex setting of Table 5, we observe similar behavior. The Laplace distribution has a variance different from 0, but still has the smallest variance. Extending the variances to RQMC (latin) in Table 6, we observe similar behavior, but substantially smaller variances.

Table 5. Variances in the case of f being the sign of the sine function ($f(x) = \text{sign}(\sin(x))$), smoothed with each respective distribution, and evaluating the variance at point 0, in dependence of the number of samples s .

$f(x) = \text{sign}(\sin(x))$	Var = 1	Lipschitz = 1
Gaussian	$0.382/s$	$0.588/s$
Logistic	$0.318/s$	$0.690/s$
Gumbel	$0.840/s$	$1.130/s$
Cauchy	n/a	$0.452/s$
Laplace	$0.093/s$	$0.144/s$
Triangular	$1.727/s$	$2.609/s$

Table 6. Variances for $f(x) = \text{sign}(x)$ with RQMC (latin) at $s = 100$ samples, which drastically reduces the variance further. As the rate is faster than $1/s$, we report it for 100 samples.

$f(x) = \text{sign}(x)$	RQMC, Var = 1	RQMC, $L = 1$
Gaussian	0.0000224	0.0000355
Logistic	0.0000011	0.0000013
Gumbel	0.0001780	0.0002008
Cauchy	n/a	0.0000041
Laplace	0.0000000	0.0000000
Triangular	0.0109100	0.0156400

F. Experimental Details

MNIST Sorting Benchmark Experiments We train for 100 000 steps at a learning rate of 0.001 with the Adam optimizer using a batch size of 100. Following the requirements of the benchmark, we use the same model as previous works (Grover et al., 2019; Cuturi et al., 2019; Petersen et al., 2021a). That is, two convolutional layers with a kernel size of 5×5 , 32 and 64 channels respectively, each followed by a ReLU and MaxPool layer; after flattening, this is followed by a fully connected layer with a size of 64, a ReLU layer, and a fully connected output layer mapping to a scalar. For each distribution and number of samples, we choose the optimal $\gamma \in \{1, 1/3, 0.1\}$.

Warcraft Shortest-Path Benchmark Experiments Following the established protocol (Vlastelica et al., 2020), we train for 50 epochs with the Adam optimizer at a batch size of 70 and an initial learning rate of 0.001. The learning rate decays by a factor of 10 after 30 and 40 epochs each. The model is the first block of ResNet18. The hyperparameter $\gamma = 1/\beta$ as specified in Figures 13 and 14.

Utah Teapot Camera Pose Optimization Experiments We initialize the pose to be perturbed by angles uniformly sampled from $[15^\circ, 75^\circ]$. The ground truth orientation is randomly sampled from the sphere of possible orientations. The ground truth camera angle is 20° , and the ground truth camera distance is uniformly sampled from $[2.5, 4]$. The initial camera distance is sampled as being uniformly offset by $[-0.5, 6]$, thus the feasible set of initial camera distance guesses lies in $[2, 10]$. The initial camera angle is uniformly sampled from $[10^\circ, 30^\circ]$. We optimize for 1 000 steps with the Adam optimizer $[(\beta_1, \beta_2) = (0.5, 0.99)]$ and the CosineAnnealingLR scheduler with an initial learning rate of 0.3. We schedule the diagonal of $\mathbf{L}$ to decay exponentially from $[0.1, 5^\circ, 5^\circ, 0.25^\circ] \cdot 10^{0.75}$ to $[0.1, 5^\circ, 5^\circ, 0.25^\circ] \cdot 10^{-1.75}$ (the dimensions are camera distance, 2 pose angles, and the camera angle). As discussed, the success criterion is finding the angle within 5° of the ground truth angle. There is typically no local minimum within 5° and it is a reliable indicator for successful alignment.

Differentiable Cryo-Electron Tomography Experiments The ground truth values of the parameters are set to 300 kV for acceleration voltage, 3 mm for the focal length, and the ground truth sample specimen is centered as $(x, y) = (0, 0)$ nm units. For reporting the RMSE metric, the acceleration voltages are normalized by a factor of 100 to ensure that all parameters vary over commensurate ranges. For the 2-parameter optimization, the feasible set of acceleration voltage varied over a range of $[0, 1000]$ kV and the feasible set of the specimen’s x -position varied over the range $[-5, 5]$. For the 4-parameter optimization, the feasible set of acceleration voltage varied over a range of $[0, 600]$ kV, the focal length ranges over $[0, 6]$ mm, the x - and y -positions range over $[-3, 3]$. We use the Adam optimizer for both experiments, with $[(\beta_1, \beta_2) = (0.5, 0.9)]$. For the MC Search baseline, we generate sets of n uniform random points in the feasible region of the parameters, generate micrographs for these random parameter tuples using the TEM simulator (Rullgård et al., 2011), and identify the parameter tuple in the set having the lowest mean squared error with respect to the ground truth image. The RMSE between this parameter tuple and the ground truth parameters is the metric for the specific set of n randomly generated values. This is repeated 20 times to obtain the mean and standard deviation of the RMSE metric at that n .

F.1. Assets

List of assets:

- The sixth platonic solid (aka. Teapotahedron or Utah tea pot) (Arvo & Kirk, 1987) [License N/A]
- Multi-digit MNIST (Grover et al., 2019), which builds on MNIST (LeCun et al., 2010) [MIT License / CC License]
- Warcraft shortest-path data set (Vlastelica et al., 2020) [MIT License]
- PyTorch (Paszke et al., 2019) [BSD 3-Clause License]
- TEM-simulator (Rullgård et al., 2011) [GNU General Public License]

F.2. Runtimes

The runtimes for sorting and shortest-path experiments are for one full training on 1 GPU. The pose optimization experiment runtimes are the total time for all 768 seeds on 1 GPU. For the TEM-simulator, we report the CPU time per simulation sample, which is the dominant and only the measureable component of the total optimization routine time. The choice of distribution, covariate, and choice of variance reduction does not have a measurable effect on training times.

- MNIST Sorting Benchmark Experiments [1 Nvidia V100 GPU]
 - Training w/ 256 samples: 65 min
 - Training w/ 1 024 samples: 67 min
 - Training w/ 2 048 samples: 68 min
 - Training w/ 8 192 samples: 77 min
 - Training w/ 32 768 samples: 118 min
- Warcraft Shortest-Path Benchmark Experiments [1 Nvidia V100 GPU]
 - Training w/ 10 samples: 9 min
 - Training w/ 100 samples: 19 min
 - Training w/ 1 000 samples: 26 min
 - Training w/ 10 000 samples: 101 min
- Utah Teapot Camera Pose Optimization Experiments [1 Nvidia A6000 GPU]
 - Optimization on 768 seeds w/ 16 samples: 25 min
 - Optimization on 768 seeds w/ 64 samples: 81 min
 - Optimization on 768 seeds w/ 256 samples: 362 min
- Differentiable Cryo-Electron Tomography Experiments [CPU: 44 Intel Xeon Gold 5118]
 - Simulator time per sample on 1 CPU core: 67 sec

G. Additional Experimental Results

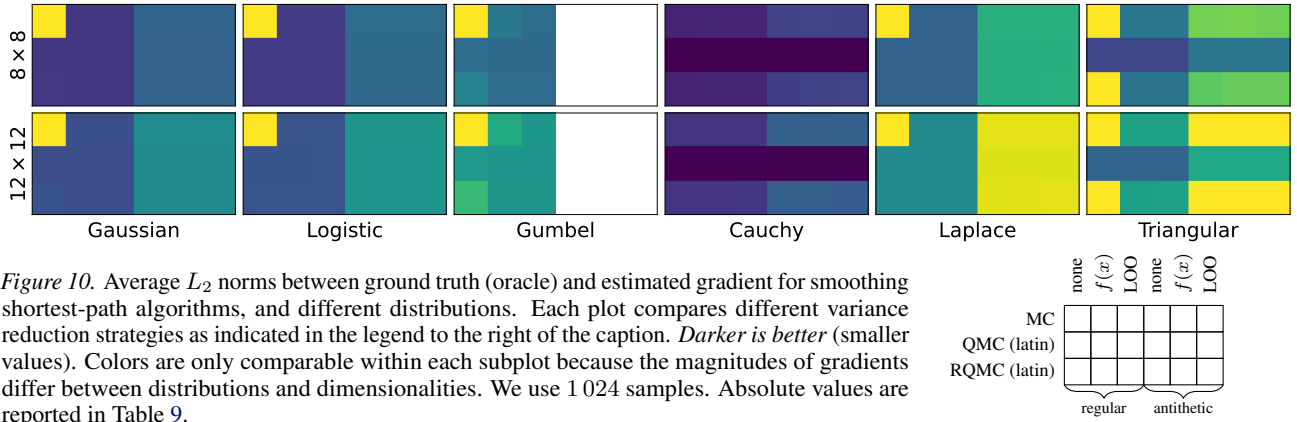


Table 7. Extension of Table 2 with additional numbers of samples and standard deviations.

Baselines		Neu.S.	Soft.S.	L. DSN	C. DSN	E. DSN	OT. S.
—		71.3	70.7	77.2	84.9	85.0	81.1
Sampling	#s	Gauss.	Logis.	Gumbel	Cauchy	Laplace	Trian.
vanilla	256	82.3±2.0	82.8±0.9	79.2±9.7	68.1±19.3	82.6±0.8	81.3±1.2
best (cv)	256	83.1±1.6	82.7±1.8	81.6±3.6	55.6±13.3	83.7±0.8	82.7±1.1
vanilla	1024	81.3±9.1	83.7±0.7	82.0±1.6	68.5±24.8	80.6±9.0	82.8±1.0
best (cv)	1024	83.9±0.6	84.0±0.5	84.2±0.6	73.0±12.6	84.3±0.6	82.4±1.6
vanilla	2048	84.1±0.6	83.6±0.8	84.0±0.5	75.7±11.6	83.8±0.7	83.2±0.6
best (cv)	2048	84.2±0.5	84.2±0.6	84.6±0.4	82.0±2.2	84.8±0.5	83.4±0.5
vanilla	8192	84.0±0.6	84.2±0.8	84.0±0.6	83.6±1.0	83.9±1.0	83.6±0.7
best (cv)	8192	84.4±0.6	84.5±0.5	84.1±0.7	84.3±0.5	84.3±0.4	83.7±0.4
vanilla	32768	84.2±0.5	84.1±0.4	84.5±0.7	84.9±0.5	84.4±0.5	83.4±0.8
best (cv)	32768	84.4±0.4	84.4±0.4	84.8±0.5	85.1±0.4	84.4±0.4	84.0±0.3

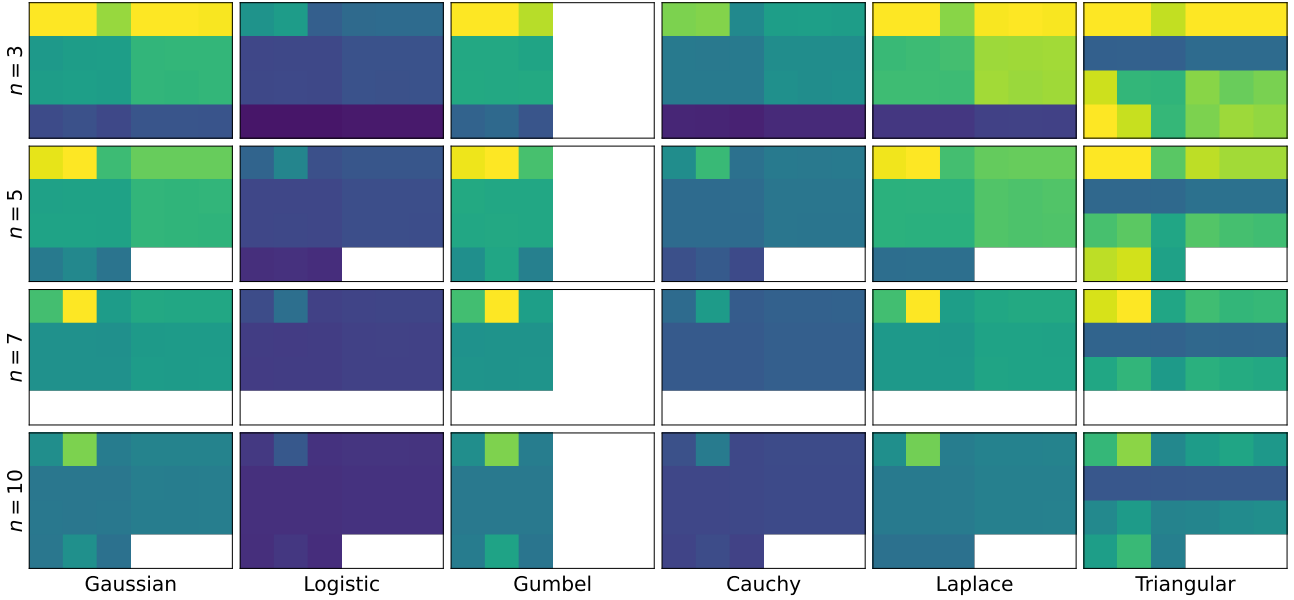


Figure 11. Average L_2 norms between ground truth (oracle) and estimated gradient for different numbers of elements to sort and rank n , and different distributions. Each plot compares different variance reduction strategies as indicated in the legend to the right of the caption. *Darker is better* (smaller values). Colors are only comparable within each subplot because the magnitudes of gradients differ between distributions and dimensionalities. We use 1 024 samples, except for Cartesian and $n = 3$ where we use $10^3 = 1\,000$ samples.

	none	$f(x)$	LOO	none	$f(x)$	LOO
MC						
QMC (latin)						
RQMC (latin)						
RQMC (cart.)						
	regular			antithetic		

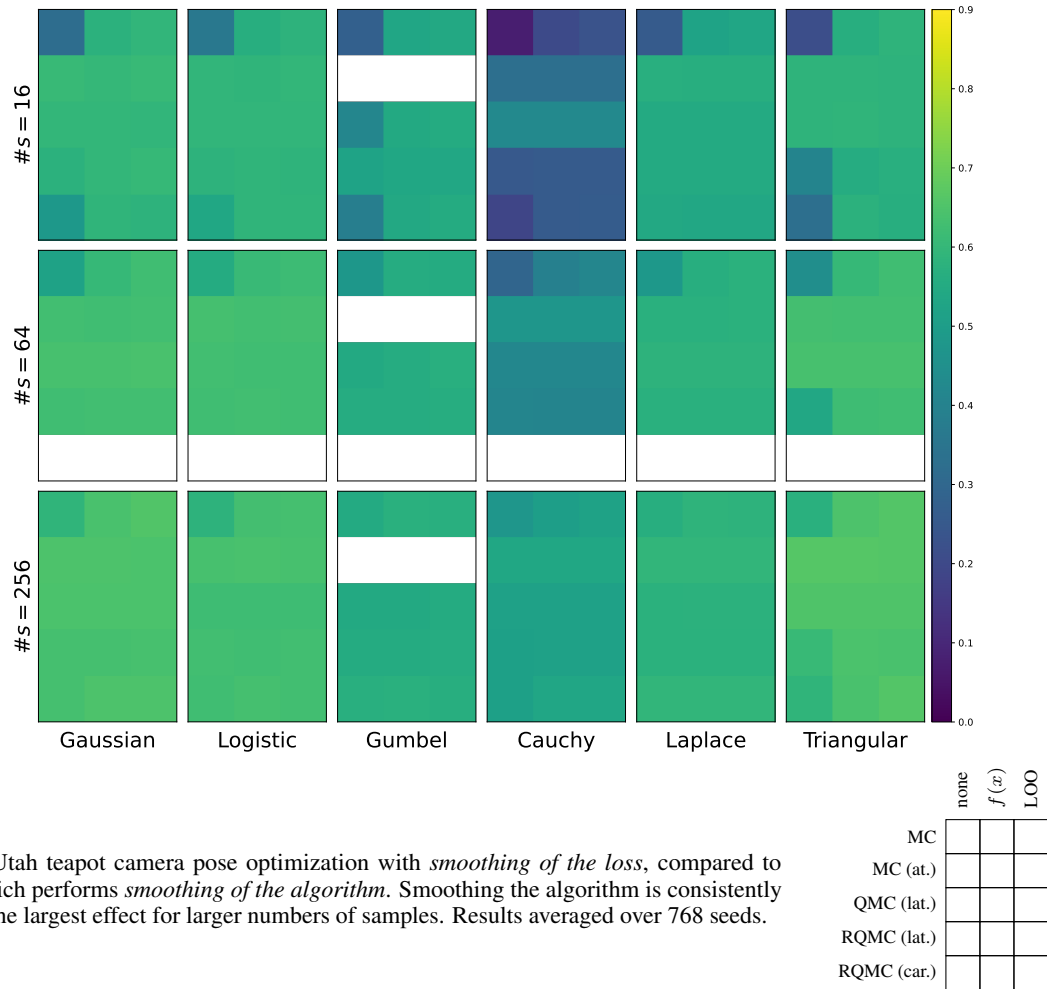


Figure 12. Utah teapot camera pose optimization with *smoothing of the loss*, compared to Figure 7, which performs *smoothing of the algorithm*. Smoothing the algorithm is consistently better, with the largest effect for larger numbers of samples. Results averaged over 768 seeds.



Figure 13. Warcraft shortest-path experiment. Left: 10 samples. Right: 100 samples. Averaged over 5 seeds. Brighter is better. Values between subplots are comparable. The displayed range is [70%, 96.5%].



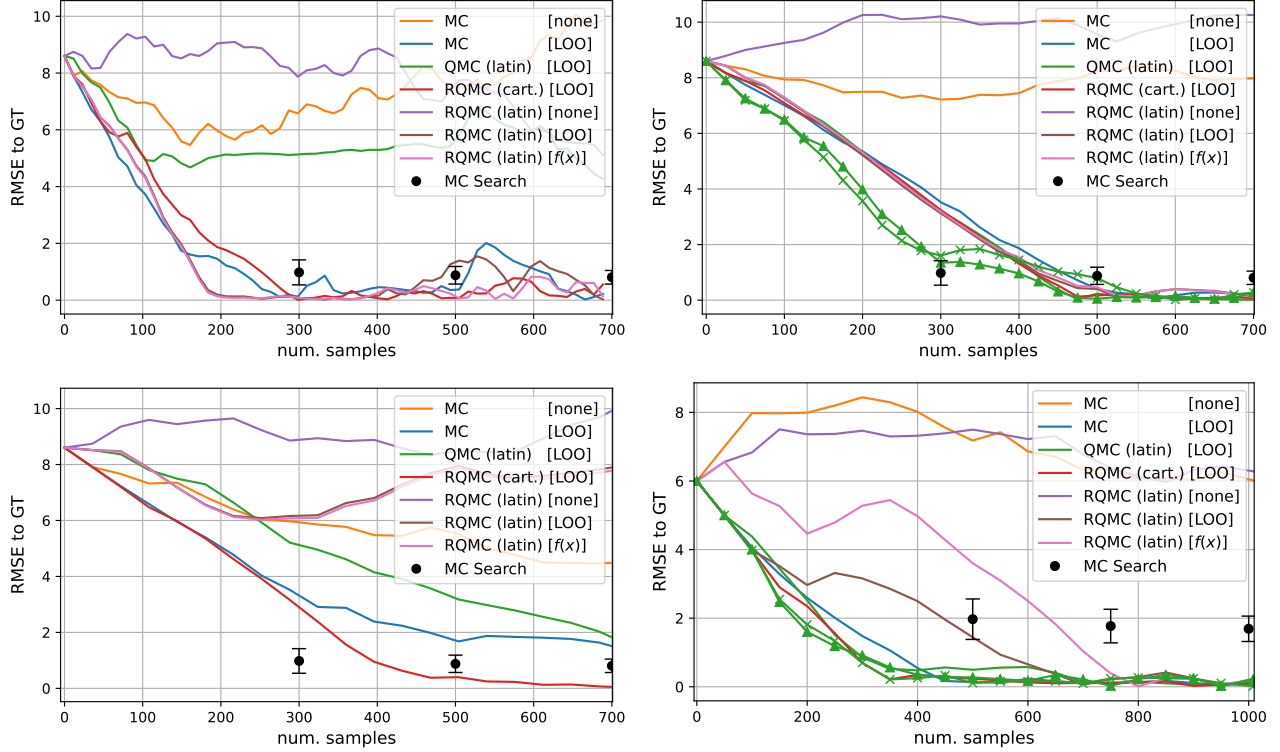


Figure 15. Cryo-Electron Tomography Experiments: RMSE with respect to Ground Truth parameters for different number of parameters optimized and for different number of samples per optimization step: (Top Left) 2-parameters & number of samples=9, (Top Right) 2-parameters & number of samples=25, (Bottom Left) 2-parameters & number of samples=36, (Bottom Right) 4-parameters. No marker lines correspond to Gaussian, $\times$ corresponds to Laplace, and $\triangle$ corresponds to Triangular distributions. Ascertaining optimal parameters with minimal evaluations is important not just for high resolution imaging, but also to minimize radiation damage to the specimen. In this light, of the covariate choices, LOO generally leads to best improvement and *none* consistently leads to deterioration in performance. The Laplace and Triangular distributions lead to best performance. For the Gaussian distribution, Cartesian RQMC is generally exhibiting best results.

Table 8. Individual absolute values from the variance simulations for differentiable sorting in Figure 3. The minimum and values within 1% of the minimum are indicated as bold.

(a) values for Gaussian ($n = 3$)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	0.0084	0.0079	0.0046	0.0055	0.0054	0.0053
QMC (lat.)	0.0029	0.0030	0.0030	0.0036	0.0036	0.0036
RQMC (l.)	0.0030	0.0030	0.0030	0.0036	0.0035	0.0036
RQMC (c.)	0.0012	0.0013	0.0012	0.0014	0.0014	0.0014
(b) values for Gaussian ($n = 5$)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	0.0241	0.0308	0.0171	0.0192	0.0192	0.0192
QMC (lat.)	0.0143	0.0144	0.0144	0.0164	0.0164	0.0164
RQMC (l.)	0.0145	0.0145	0.0144	0.0164	0.0164	0.0162
RQMC (c.)	0.0103	0.0116	0.0097	—	—	—
(c) values for Logistic ($n = 3$)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	0.0028	0.0030	0.0016	0.0019	0.0019	0.0019
QMC (lat.)	0.0012	0.0012	0.0012	0.0014	0.0014	0.0014
RQMC (l.)	0.0012	0.0012	0.0012	0.0014	0.0013	0.0014
RQMC (c.)	0.0003	0.0003	0.0003	0.0004	0.0004	0.0004
(d) values for Logistic ($n = 5$)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	0.0081	0.0114	0.0061	0.0067	0.0067	0.0067
QMC (lat.)	0.0053	0.0053	0.0054	0.0060	0.0060	0.0060
RQMC (l.)	0.0053	0.0054	0.0053	0.0060	0.0060	0.0059
RQMC (c.)	0.0033	0.0036	0.0033	—	—	—
(e) values for Gumbel ($n = 3$)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	0.0086	0.0082	0.0048	—	—	—
QMC (lat.)	0.0033	0.0033	0.0032	—	—	—
RQMC (l.)	0.0033	0.0033	0.0033	—	—	—
RQMC (c.)	0.0017	0.0018	0.0014	—	—	—
(f) values for Gumbel ($n = 5$)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	0.0243	0.0323	0.0177	—	—	—
QMC (lat.)	0.0151	0.0149	0.0150	—	—	—
RQMC (l.)	0.0150	0.0151	0.0150	—	—	—
RQMC (c.)	0.0124	0.0148	0.0109	—	—	—
(g) values for Cauchy ($n = 3$)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	0.0043	0.0044	0.0026	0.0030	0.0030	0.0030
QMC (lat.)	0.0022	0.0022	0.0022	0.0027	0.0027	0.0027
RQMC (l.)	0.0022	0.0022	0.0022	0.0027	0.0026	0.0027
RQMC (c.)	0.0006	0.0006	0.0005	0.0006	0.0006	0.0006
(h) values for Cauchy ($n = 5$)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	0.0123	0.0169	0.0094	0.0102	0.0101	0.0102
QMC (lat.)	0.0088	0.0087	0.0088	0.0098	0.0098	0.0098
RQMC (l.)	0.0088	0.0088	0.0087	0.0098	0.0097	0.0097
RQMC (c.)	0.0061	0.0070	0.0056	—	—	—
(i) values for Laplace ($n = 3$)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	0.0086	0.0074	0.0044	0.0054	0.0054	0.0054
QMC (lat.)	0.0037	0.0037	0.0038	0.0046	0.0046	0.0047
RQMC (l.)	0.0037	0.0037	0.0037	0.0047	0.0046	0.0046
RQMC (c.)	0.0009	0.0009	0.0009	0.0010	0.0011	0.0010
(j) values for Laplace ($n = 5$)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	0.0245	0.0305	0.0176	0.0191	0.0192	0.0192
QMC (lat.)	0.0159	0.0160	0.0160	0.0182	0.0180	0.0182
RQMC (l.)	0.0160	0.0159	0.0159	0.0182	0.0181	0.0181
RQMC (c.)	0.0091	0.0091	0.0091	—	—	—
(k) values for Triangular ($n = 3$)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	0.1191	0.0683	0.0490	0.0659	0.0624	0.0602
QMC (lat.)	0.0166	0.0169	0.0166	0.0189	0.0188	0.0188
RQMC (l.)	0.0498	0.0358	0.0352	0.0444	0.0417	0.0431
RQMC (c.)	0.0682	0.0494	0.0361	0.0435	0.0461	0.0452
(l) values for Triangular ($n = 5$)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	0.3329	0.2779	0.1857	0.2255	0.2157	0.2149
QMC (lat.)	0.0844	0.0845	0.0851	0.0932	0.0931	0.0928
RQMC (l.)	0.1768	0.1872	0.1479	0.1827	0.1765	0.1737
RQMC (c.)	0.2251	0.2325	0.1430	—	—	—

Table 9. Individual absolute values from the variance simulations for differentiable shortest-paths in Figure 10. The minimum and values within 1% of the minimum are indicated as bold.

(a) values for Gaussian (8×8)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	1330.01	4.17	4.17	8.32	8.32	8.34
QMC (lat.)	4.04	4.04	4.04	8.04	8.04	8.07
RQMC (l.)	4.25	4.05	4.05	8.10	8.09	8.12
(b) values for Gaussian (12×12)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	6800.98	20.93	20.95	41.82	41.78	41.88
QMC (lat.)	20.60	20.60	20.65	41.12	41.11	41.18
RQMC (l.)	21.69	20.66	20.68	41.31	41.33	41.42
(c) values for Logistic (8×8)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	1449.44	4.53	4.53	9.04	9.04	9.05
QMC (lat.)	4.42	4.42	4.43	8.80	8.80	8.83
RQMC (l.)	4.44	4.44	4.44	8.88	8.87	8.90
(d) values for Logistic (12×12)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	7447.38	22.83	22.86	45.62	45.61	45.75
QMC (lat.)	22.56	22.56	22.61	45.01	44.99	45.07
RQMC (l.)	22.66	22.65	22.68	45.30	45.32	45.41
(e) values for Gumbel (8×8)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	2275.31	10.35	9.08	—	—	—
QMC (lat.)	9.11	8.84	8.85	—	—	—
RQMC (l.)	11.33	8.91	8.91	—	—	—
(f) values for Gumbel (12×12)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	11642.74	52.89	46.11	—	—	—
QMC (lat.)	46.88	45.41	45.48	—	—	—
RQMC (l.)	58.12	45.74	45.80	—	—	—
(g) values for Cauchy (8×8)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	249027.67	263426.66	255440.59	507004.19	525973.88	509764.25
QMC (lat.)	2533.24	2532.93	2537.32	2531.24	2532.92	2537.35
RQMC (l.)	251018.28	267124.91	264146.84	476293.00	507766.00	529030.06
(h) values for Cauchy (12×12)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	1316801.88	1284078.38	1297748.25	2657888.00	2631427.25	2633413.50
QMC (lat.)	12922.79	12922.31	12948.75	12931.28	12928.22	12945.27
RQMC (l.)	1318297.38	1299869.75	1365709.75	2606723.50	2615697.50	2529304.00
(i) values for Laplace (8×8)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	2641.38	8.15	8.15	16.28	16.27	16.29
QMC (lat.)	8.04	8.05	8.06	16.01	16.00	16.04
RQMC (l.)	8.09	8.09	8.10	16.19	16.17	16.22
(j) values for Laplace (12×12)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	13593.82	41.40	41.45	82.73	82.71	82.92
QMC (lat.)	41.06	41.07	41.16	81.78	81.75	81.92
RQMC (l.)	41.32	41.31	41.36	82.62	82.64	82.80
(k) values for Triangular (8×8)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	3090.80	10.21	10.11	20.27	20.43	20.07
QMC (lat.)	5.57	5.57	5.57	10.17	10.18	10.20
RQMC (l.)	884.22	9.88	9.82	19.14	19.71	19.76
(l) values for Triangular (12×12)						
	none	$f(x)$	LOO	none	$f(x)$	LOO
	regular			antithetic		
MC	15975.60	49.73	49.89	99.81	99.32	100.31
QMC (lat.)	28.28	28.28	28.34	51.79	51.79	51.86
RQMC (l.)	4606.71	49.01	49.47	98.56	98.66	98.01